

EKF Localization & Mapping for a Formula Student Driverless Car

A beginner-friendly guide to the Extended Kalman Filter, and how I implemented one in ROS

Yasser Galal

Formula Student 2023

Table of Contents

About this document

This document explains how I built an **Extended Kalman Filter (EKF)** that lets a Formula Student driverless race car figure out *where it is* on a track it has never seen before, using nothing but the cones placed around that track.

Who this is for. You do not need a background in robotics, statistics, or control theory. If you remember roughly what an average is, what a square root is, and what a derivative is, you can read this document from start to finish. Everything else I will explain as we go.

How I have tried to write it. Plainly. Every time I introduce a symbol I say out loud what it means. Every time I introduce an equation I explain what each piece of it does and *why* it is there before I ever ask you to use it. Where a formula would be easier to understand as a sentence, I write the sentence too.

How to read this

The document has three parts, and they build on each other:

Part	Chapters	What it covers
Part I — The maths you actually need	1–9	Averages, spread, Gaussians, noise, covariance. No filters yet, just the vocabulary.

Part	Chapters	What it covers
Part II — The filters	10–17	The Kalman filter, why it breaks on real robots, and how the EKF fixes it.
Part III — The implementation	18–21	The actual ROS code, line by line, plus how to run and tune it.

If you already know your statistics, start at Chapter 10. If you only want to see the robot code, start at Chapter 18 — but I would honestly read Chapter 16 first, because that is where the equations you will see in the code are explained.

At the very back there are three appendices: a **notation table** (Appendix A) that you can flip back to whenever a symbol confuses you, a **complete worked example** (Appendix B) where I run one full cycle of the filter by hand with real numbers, and a **glossary** (Appendix C).

Part I — The maths you actually need

1. Introduction: the problem I was trying to solve

1.1 The situation

In Formula Student Driverless, a small race car has to drive around a track marked out by traffic cones, with nobody in it. On the first lap the car has never seen the track before. It has to work out the shape of the track *while* driving around it.

To drive safely the car needs to answer two questions at every instant:

1. **Where am I?** (its position and which way it is pointing)
2. **Where is everything else?** (where the cones are)

Answering the first question alone is called **localization**: finding your position in a map you were given. Answering both at the same time is called **SLAM** — *Simultaneous Localization And Mapping*. The word “simultaneous” is doing real work there: the car has to build the map and locate itself inside that same map at the same time, which sounds circular, and in a sense it is. That is exactly what makes the problem interesting.

What this document describes is **EKF-SLAM**: the cone positions are estimated by the filter and stored inside its state, not handed to it in advance.

1.2 Why this is hard

The obvious approach is: measure how fast the wheels are turning, measure how much the car has rotated, and add it all up over time. This is called **dead reckoning**, and it works — for about ten seconds.

The problem is that every measurement has a small error in it, and when you add measurements up over time you add the errors up too. The errors never cancel out neatly; they accumulate. This is called **drift**. After a minute of driving, a dead-reckoning estimate can be metres away from the truth, and on a track that is only three metres wide, metres of error means you hit a cone.

So we need something that does not just add measurements up. We need something that **combines** different sources of information and weighs each one according to how much it deserves to be trusted. That is exactly what a Kalman filter is.

1.3 Why the Extended Kalman Filter

There are several algorithms that solve SLAM. The main families are:

- **EKF-SLAM** — the one used here. Represents everything with a single Gaussian. Simple, fast for small maps, well understood.
- **Particle-filter SLAM (FastSLAM)** — represents position as a cloud of thousands of guesses. Handles ambiguity better, costs more compute.
- **Graph-based SLAM** — builds a graph of poses and constraints and optimises the whole thing at once. The most accurate family, and what most modern systems use, but the heaviest.
- **Unscented Kalman Filter (UKF)** — like the EKF but handles non-linearity more cleverly (see §16.5).

I chose the EKF for three reasons: our map was small (a few dozen cones), our compute budget was small, and it is the algorithm that teaches you the most about how estimation actually works. Almost everything in the other methods is a variation on ideas that appear in the EKF.

That choice is not just convenience. Le Large's 2020 Master's thesis at KIT compared EKF-SLAM against GraphSLAM on data recorded from a real Formula Student driverless car, measured against high-precision DGPS ground truth. He

found that GraphSLAM produced more accurate maps — but that it “drained the computational resources much faster than EKF SLAM”, and concluded that where compute is constrained, EKF-SLAM remains a valid choice whose lower accuracy may be repaid by lower CPU usage. That is precisely the trade-off we faced, and I return to it in §21.4.

1.4 The one-paragraph summary of the whole document

If you read nothing else, read this. The filter keeps a **guess** about where the car is, together with a number describing **how confident it is** in that guess. Every cycle it does two things. First it **predicts**: “I was here, I have been driving at this speed for this long, so I am probably now over there” — and because prediction involves guessing, its confidence gets *worse*. Then it **updates**: “I can see a cone, and it is 4.1 metres away — but if my prediction were right, that cone should be 3.83 metres away, so my prediction was slightly off, let me correct it” — and because a measurement adds information, its confidence gets *better*. Predict, update, predict, update, forever. Everything else in this document is detail about how to do those two steps properly.

2. Mean and expected value

2.1 Nothing measures perfectly

Nothing in the physical world is perfectly executed or perfectly measured. If you command a robot to move exactly 1 metre forward, it will move 1.03 metres, or 0.97 metres. If you measure a temperature five times you will get five slightly different answers. This happens because of imperfect sensors, vibration, temperature changes, electrical interference, wheels slipping, and a hundred other things.

Because we can never get the exact truth, estimation is not about finding the true value. It is about finding the *best possible guess* about the true value, and — just as importantly — being honest about how good that guess is.

2.2 A worked example: measuring the force of a punch

Suppose we want to know the force of a punch. We have a physical model:

$$F = m \times a$$

where F is force in newtons, m is mass in kilograms and a is acceleration in metres per second squared. We plug in our numbers and calculate **550 N**.

We also have a sensor, and it reads **530.7 N**.

Which one is right? We do not know. But notice something: our model neglected air resistance, the flex of the target, and everything else. The sensor at least measured the real world, including all those effects. So we would probably trust the sensor a little more — but not completely, because sensors have errors too.

Now suppose we do not have one sensor but five, all measuring the same punch:

Sensor	Reading (N)
1	530.7
2	531.2
3	530.1
4	535.4
5	560.8

Which one is the true value? Now we really cannot say. But we can do something sensible: take the **average**.

$$\mu = \frac{530.7 + 531.2 + 530.1 + 535.4 + 560.8}{5} = \frac{2688.2}{5} = 537.64 \text{ N}$$

The Greek letter μ (“mu”) is the standard symbol for the mean, and I will use it throughout this document.

Why is averaging a good idea? Because errors tend to be random, and random errors point in different directions. Some readings are too high, some are too low. When you add them together, the too-high ones partly cancel the too-low ones, and what is left over is closer to the truth than any single reading. This is not a trick — it is a mathematical fact, and it is the single idea that the entire Kalman filter is built on. *Combining measurements beats trusting one measurement.*

2.3 Mean versus expected value

These two terms are closely related and often used interchangeably, but there is a real difference worth knowing.

The mean (also called the *sample mean* or *average*) is what you get by adding up numbers you actually have and dividing by how many there are. It is a fact about data in front of you.

For example: I have five 25-cent coins and three 1-dollar notes. What is the average value of a piece of currency in my pocket?

$$\mu = \frac{25+25+25+25+25+300+300+300}{8} = \frac{1025}{8} = 128.125 \text{ cents}$$

Nothing uncertain about that. I counted, I divided, I am done.

The expected value, written $E[x]$, is the average you would *expect to see* if you repeated a random experiment forever. It is a property of the underlying process, not of the data you happen to have collected. The five force readings above are five samples drawn from a process; $E[F]$ is the value you would converge to if you could take infinitely many readings.

Here is the connection, and it is the useful part: **the sample mean is our best estimate of the expected value**. We cannot take infinitely many readings, so we take five, average them, and use that as our estimate of $E[F]$. The more readings we take, the closer our sample mean gets to the true expected value.

For everything we do in this document we will use μ for both, because in practice the distinction will not trip us up. It is worth knowing it exists, though, because in a statistics textbook it matters.

So: for our punch, our best estimate of the true force is **537.64 N**. But how much should we trust that number? That is the next chapter.

3. Variance and standard deviation

3.1 A number is not enough

We estimated the force as 537.64 N. But look again at the readings: 530.7, 531.2, 530.1, 535.4, 560.8. Four of them cluster tightly around 531, and then one is way out at 560.8. Our average of 537.64 does not sit near *any* of the actual readings.

Compare that with a different imaginary set of readings: 537.5, 537.6, 537.7, 537.6, 537.6. These also average to about 537.6 — but this time we would feel much more confident, because all the readings agree with each other.

Same average, completely different levels of trust. So the average alone does not tell us enough. **We need a second number that describes how spread out the data is**. That number is the variance.

3.2 The variance

The **variance**, written σ^2 (“sigma squared”), measures how far the data points are spread from their mean. It is calculated like this:

$$\sigma^2 = \frac{\sum_{i=1}^N (x_i - \mu)^2}{N - 1}$$

Read out loud, that says: *for every measurement, find how far it is from the mean; square that distance; add up all the squares; divide by one less than the number of measurements.* The squares and the summation sign both matter — leave either out and the formula gives the wrong answer.

Where:

- x_i — the i -th measured value
- μ — the mean of all the values
- N — how many values there are
- Σ — the summation symbol, meaning “add up all of these”

Why square the differences? Two reasons. First, if we did not square them, the positive and negative differences would cancel out and we would always get zero — which is useless. Second, squaring punishes large errors much more than small ones: being 10 away contributes 100, but being 1 away contributes only 1. That matches our intuition that one wildly wrong reading is a bigger problem than several slightly wrong ones.

Why divide by $N - 1$ and not N ? This is called *Bessel’s correction*. The short version: we used the data itself to calculate μ , so the data is “artificially close” to μ , and dividing by N would make the spread look smaller than it really is. Dividing by $N - 1$ compensates. Use $N - 1$ when your data is a *sample* from a larger population (which is nearly always the case with sensors), and N when you genuinely have the entire population.

3.3 The standard deviation

The variance has an awkward property: because we squared everything, its units are squared too. Our forces are in newtons, so our variance is in newtons-squared, which is not a physically meaningful thing.

The fix is to take the square root. That gives the **standard deviation**, written σ :

$$\sigma = \sqrt{\sigma^2}$$

The standard deviation is back in the original units — newtons — so we can compare it directly to the measurements. This makes it far more intuitive than the variance, and it is usually the number engineers quote to each other.

3.4 Working it out for the punch example

Our readings: 530.7, 531.2, 530.1, 535.4, 560.8, with $\mu=537.64$ N.

First, the difference of each reading from the mean, and its square:

Reading x_i	$x_i - \mu$	$(x_i - \mu)^2$
530.7	-6.94	48.1636
531.2	-6.44	41.4736
530.1	-7.54	56.8516
535.4	-2.24	5.0176
560.8	+23.16	536.3856
Sum		687.892

Now divide by $N - 1 = 4$:

$$\sigma^2 = \frac{687.892}{4} = 171.973 \text{ N}^2$$

And take the square root:

$$\sigma = \sqrt{171.973} = 13.114 \text{ N}$$

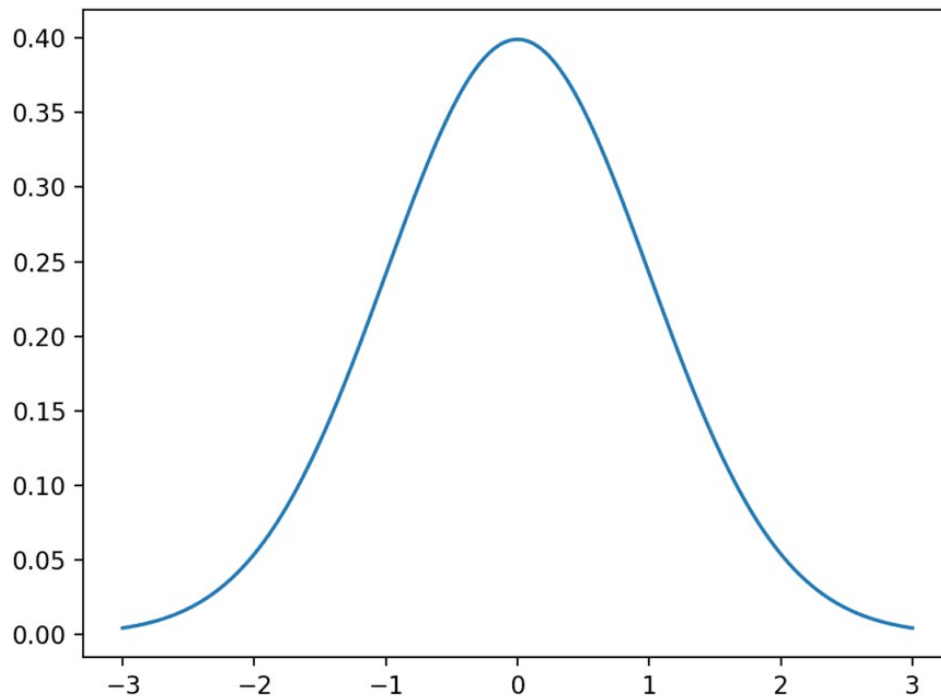
What does that actually mean? It means a typical reading from this set of sensors lands about 13 newtons away from the average. Our estimate is “537.64 N, give or take about 13 N”. That is a much more honest and useful statement than “537.64 N” on its own.

Notice how the single bad reading of 560.8 N dominates the sum — it contributed 536 of the 688 total, which is 78% of all the spread from one reading out of five. That is the squaring at work, and it is worth remembering: a single outlier can badly damage an estimator that assumes everything is well behaved.

4. The Gaussian distribution

4.1 The shape almost everything makes

We now have two numbers describing our measurements: the mean μ (where they cluster) and the variance σ^2 (how spread out they are). It turns out that if you plot enough real-world measurements on a graph, they almost always make the same characteristic bell shape:



The Gaussian distribution, also called the normal distribution or probability density function

This shape is called the **Gaussian distribution**, the **normal distribution**, or the **probability density function (PDF)**. All three names mean the same thing, and I will use them interchangeably because you will meet all three in the literature.

Its formula is:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

The formula for the Gaussian distribution

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Do not worry about this formula. We are never going to evaluate it in this document. I include it only so you recognise it when you see it elsewhere. What

matters is that the formula contains exactly two adjustable quantities: μ and σ . That is the crucial insight — **a Gaussian is completely described by just two numbers**. Tell me the mean and the variance and I can draw you the entire curve. This is why Kalman filters are fast: instead of tracking a complicated probability shape, we only ever have to track two numbers (or in higher dimensions, a vector and a matrix).

A note on the name. The distribution is named after Carl Friedrich Gauss, who used it in 1809 for analysing astronomical data. He did not actually discover it — Abraham de Moivre described it in 1733, and Pierre-Simon Laplace developed it further. Gauss's name stuck because his application of it was so influential.

4.2 Why does everything look like this?

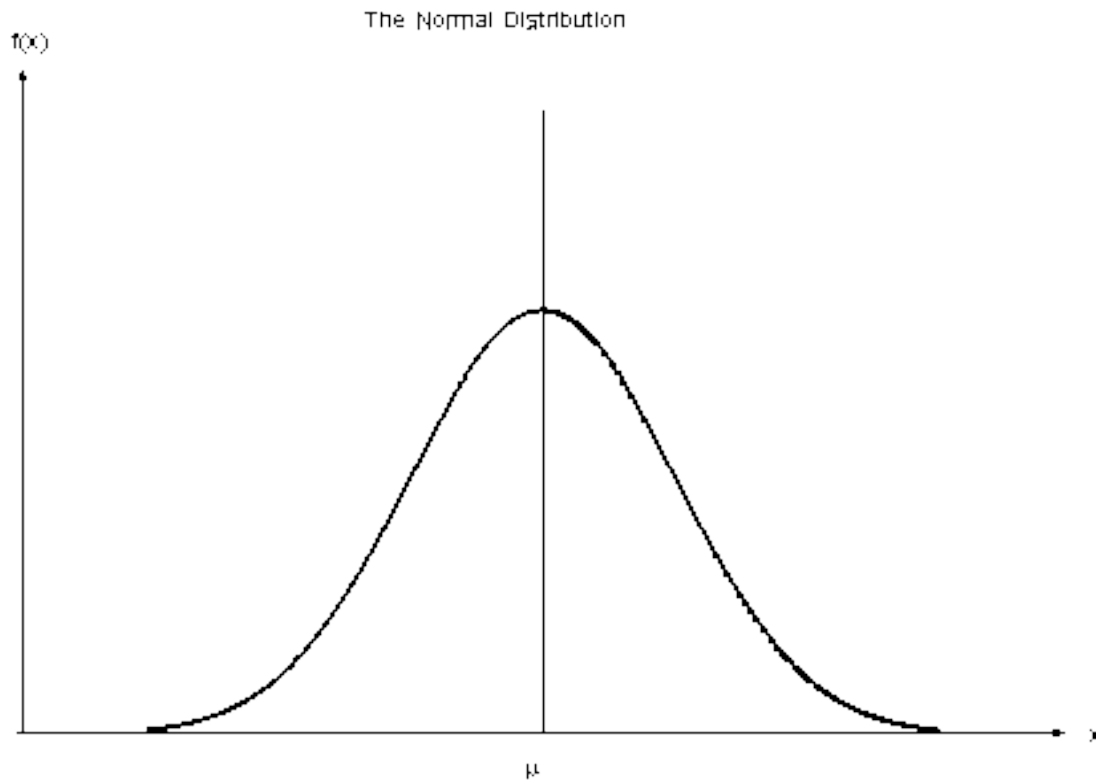
This is a fair question and it has a genuinely beautiful answer, called the **Central Limit Theorem**. Roughly, it says: *if a quantity is affected by many small independent random influences added together, the result will be Gaussian — no matter what the individual influences look like.*

Think about what makes a sensor reading wrong: thermal noise in the electronics, a bit of vibration, a small calibration offset, quantisation in the analogue-to-digital converter, electromagnetic interference, air currents. Dozens of tiny independent effects, added up. The Central Limit Theorem says the total is Gaussian. That is why this one shape appears everywhere in engineering, and it is why assuming Gaussian noise is usually a reasonable thing to do.

4.3 Reading information off the curve

We do not need to model the curve or evaluate its formula. We just need to read three things off it.

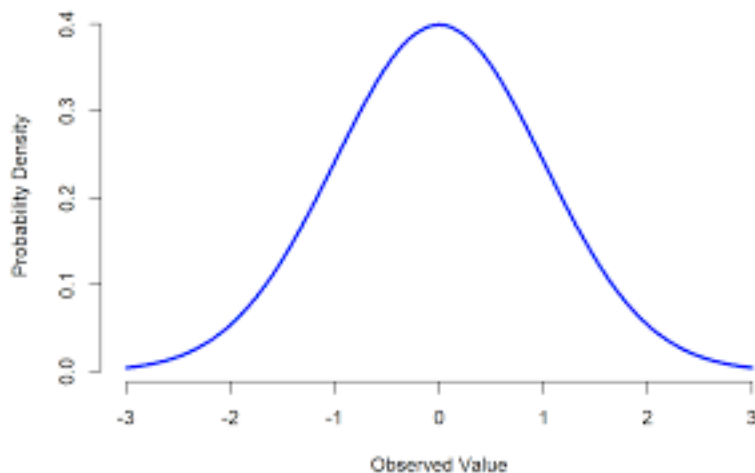
First, the mean — the centre of the curve, its highest point:



The mean is the centre of the distribution

This is our best single guess. It is the value we would report if someone forced us to give exactly one number.

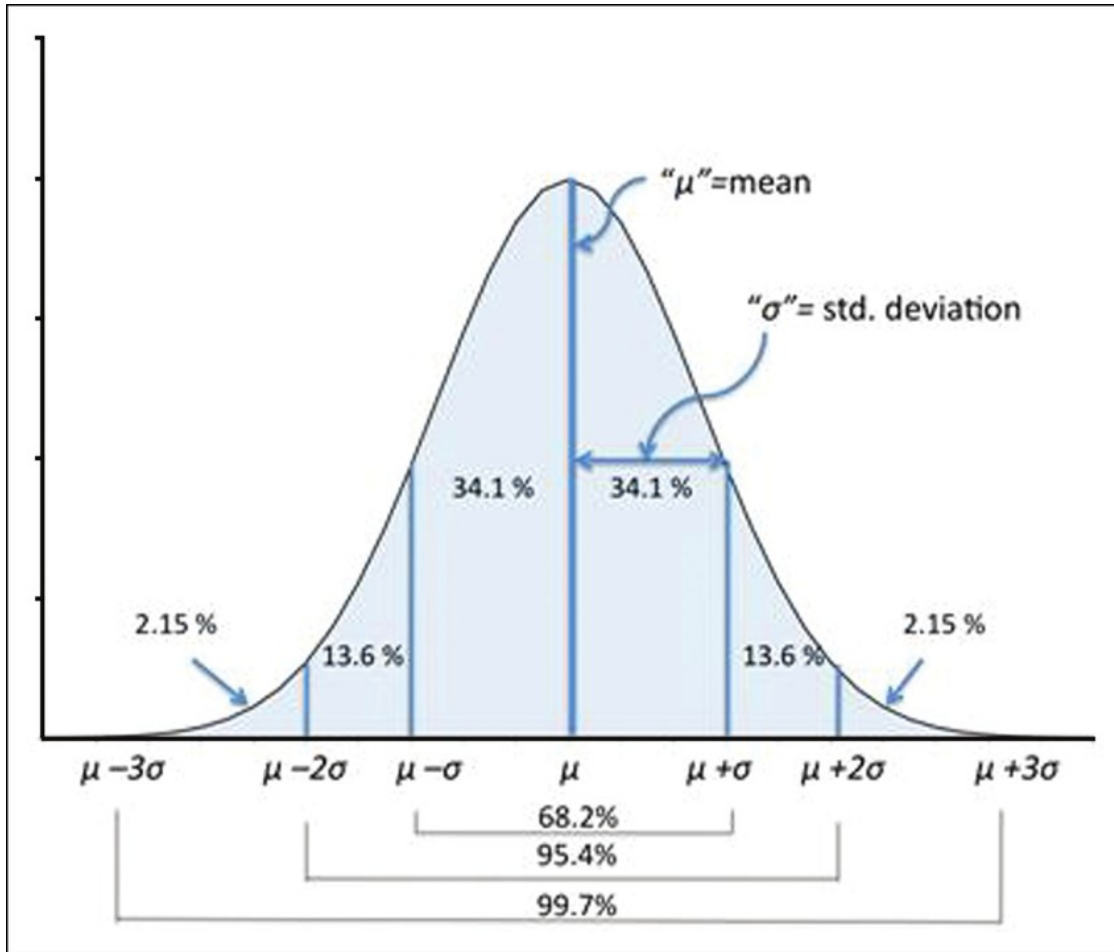
Second, the variance — how wide the curve is:



The variance describes the width of the distribution

A wide curve means the measurements were scattered and we are not very sure.
A narrow curve means they agreed with each other and we are confident.

Third, the standard deviation — the horizontal distance from the centre out to where the curve changes from bending downwards to bending outwards:

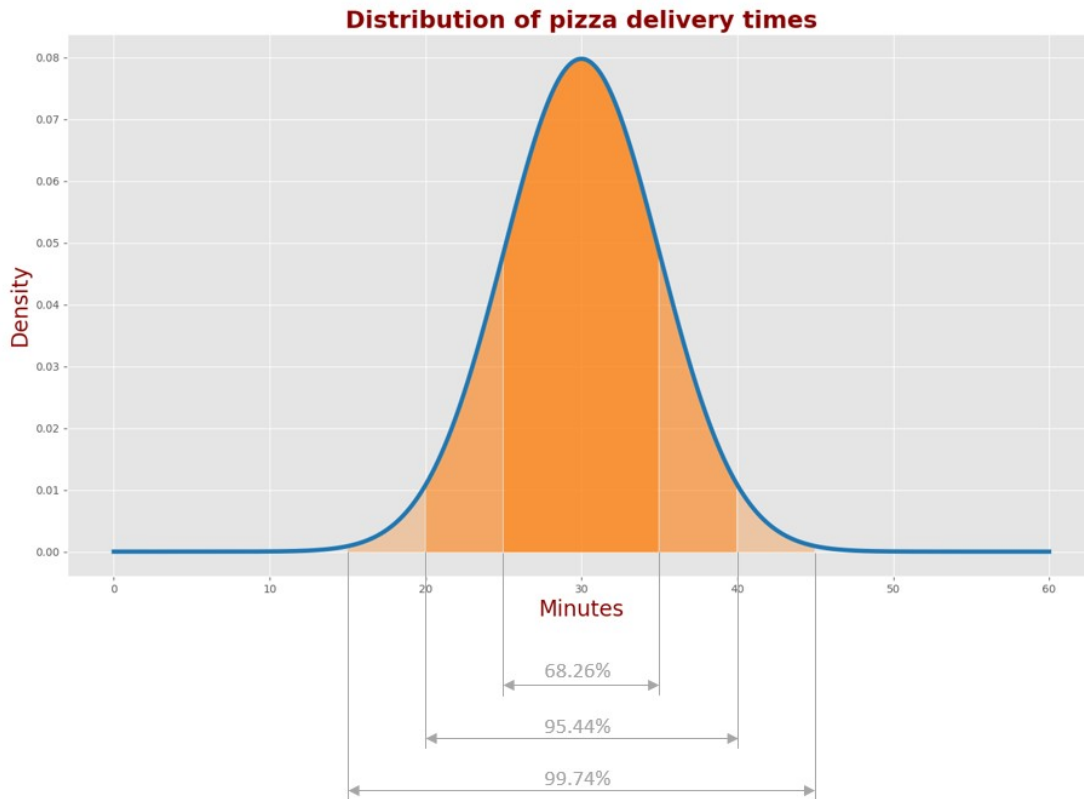


The standard deviation measured from the centre of the curve

4.4 The 68-95-99.7 rule

Here is the single most useful fact about Gaussians, and it is worth memorising:

- **68.26%** of all measurements land within $\mu \pm 1\sigma$
- **95.44%** of all measurements land within $\mu \pm 2\sigma$
- **99.74%** of all measurements land within $\mu \pm 3\sigma$



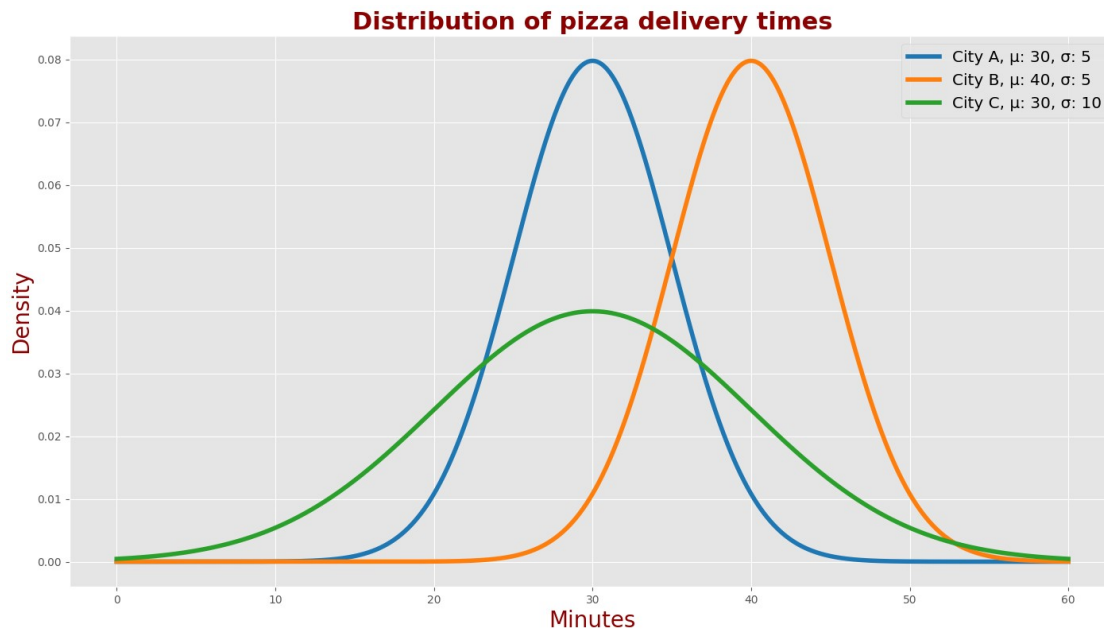
The proportions of the normal distribution

This is what turns the standard deviation from an abstract number into something you can actually use. It lets you make statements like *“I am 95% sure the true value is between these two numbers”* — which is exactly the kind of statement an engineer needs to make.

Applied to our punch: $\mu=537.64$ N and $\sigma=13.11$ N, so we can say we are about 95% confident the true force is between $537.64 - 26.2=511.4$ N and $537.64+26.2=563.9$ N. That is a wide range, which honestly reflects the fact that our sensors disagreed with each other quite a lot.

4.5 A second example: pizza delivery

Statistics is much easier when it is about something other than robots, so here is a completely different example. Three cities, and we record how long pizza delivery takes in each:



Delivery time distributions for three cities

- **City A:** mean 30 minutes, standard deviation 5 minutes
- **City B:** mean 40 minutes, standard deviation 5 minutes
- **City C:** mean 30 minutes, standard deviation 10 minutes

Compare **A and B**: the two curves are exactly the same shape, just shifted. Deliveries in city A take 10 minutes less on average, but the *reliability* is identical — both cities are equally predictable, one is just faster.

Compare **A and C**: the two curves are centred in the same place but have different widths. On average you wait the same time in both, but city C is much less predictable. Sometimes your pizza arrives very quickly, sometimes you wait a long time.

This is the distinction that matters for a robot too: *where* the estimate is, and *how reliable* it is, are two separate things and you need to track both.

Applying the 68–95–99.7 rule to city A ($\mu=30$, $\sigma=5$):

- 68.26% of deliveries take between **25 and 35** minutes ($\mu \pm \sigma$)
- 95.44% of deliveries take between **20 and 40** minutes ($\mu \pm 2\sigma$)
- 99.74% of deliveries take between **15 and 45** minutes ($\mu \pm 3\sigma$)

So if your pizza is 50 minutes late in city A, something has genuinely gone wrong — that is beyond 3σ , which happens roughly once in 370 deliveries.

4.6 Two names you will see

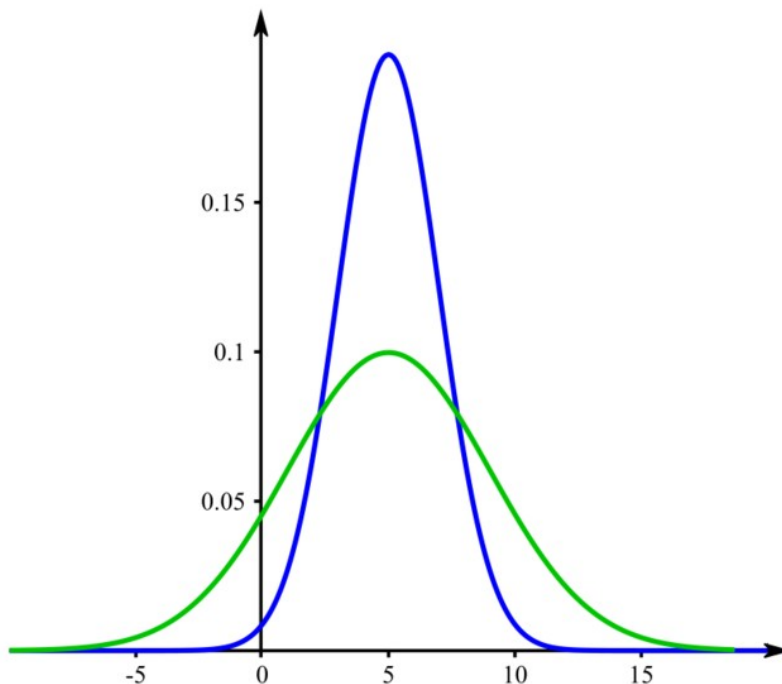
Because they get used a lot in the Kalman filter literature:

- The **standard deviation** is often called the **measurement uncertainty**, because it tells you how uncertain you are about where the true value sits.
- The **variance** is often called the **measurement error** or **covariance**, because it describes the whole region the true value could be hiding in.

From Chapter 9 onwards I will mostly use the single word **covariance** to cover all of these, because that is the word the code uses.

5. Wide and narrow Gaussians

Now that we can read a Gaussian, here is the question that makes them useful: **which is better, a wide one or a narrow one?**



A narrow distribution (blue) and a wide distribution (green)

The green curve is much more spread out than the blue one, so the green curve has a higher variance. High variance means high uncertainty — the true value could be almost anywhere under that wide curve. Low variance means low uncertainty — the true value is pinned down in a narrow band.

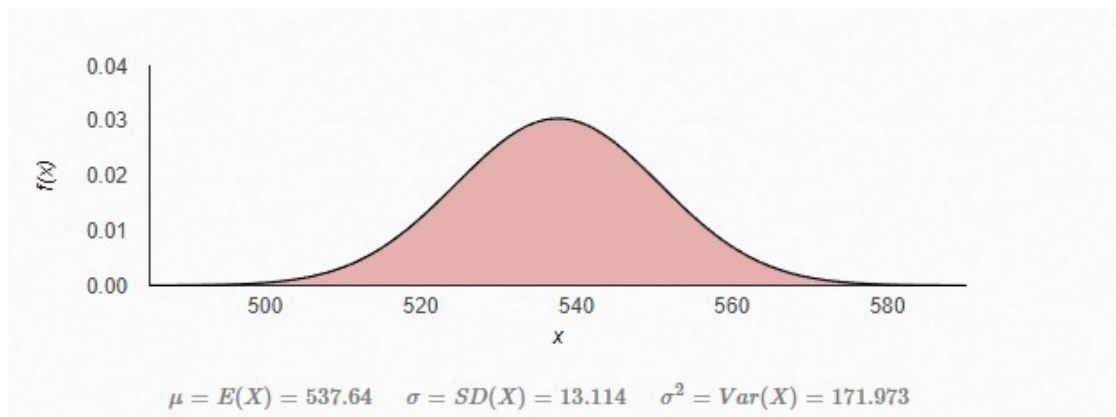
So narrow is better. Always. If you have one measurement with low variance and another with high variance, the low-variance one is more valuable, and any sensible filter should trust it more.

Remember that sentence, because it is the entire idea behind the Kalman gain in Chapter 11. When the filter has to decide how much to trust a measurement versus how much to trust its own prediction, the rule it uses is exactly this: *trust the narrower one more.*

The punch example as a Gaussian

We only need μ and σ^2 to draw the curve, and we have both:

$$\mu = 537.64 \text{ N} \quad \sigma^2 = 171.973 \text{ N}^2 \quad \sigma = 13.114 \text{ N}$$



The distribution of our force measurements

This is quite a wide curve. Our sensors were not in good agreement, so our estimate has a lot of uncertainty in it. If this were a real engineering problem, the honest conclusion would be “we need better sensors, or many more readings” — and in Chapter 12 we will see exactly how much a filter can rescue a situation like this.

6. Accuracy and precision

These two words get used interchangeably in ordinary English, but in engineering they mean genuinely different things, and confusing them will cause you real problems.

6.1 Accuracy

Accuracy is how close your measurements are to the true value.

If the true force on a body is 101 N and we measure 96 N, then our accuracy is:

$$\frac{96}{101} \times 100 = 95.05\%$$

Accuracy relates to the **mean** of a Gaussian: an accurate sensor has its curve centred on the true value. An inaccurate sensor has its curve centred somewhere else — this systematic offset is called **bias**.

6.2 Precision

Precision is how close your measurements are to each other, regardless of whether they are near the truth.

Here are two sensors measuring the same thing:

	Readings			
Sensor A	99.2, 99.1, 99.5, 99.9, 99.0, 99.4			
Sensor B	82.2, 95.1, 90.5, 85.3, 91.0, 75.2			
Accurate Precise	Not Accurate Precise	Accurate Not Precise	Not Accurate Not Precise	
				

Comparing the consistency of two sensors

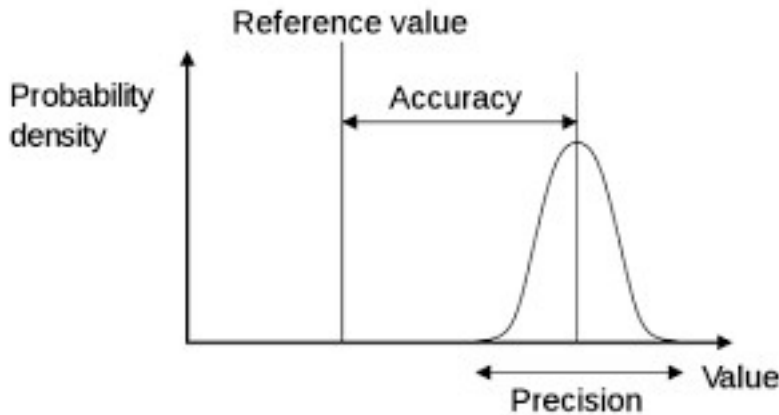
Sensor A is far more **precise** — its readings are tightly clustered, all within 0.9 of each other. Sensor B's readings are scattered across 20 units.

Precision relates to the **variance** of a Gaussian:

High precision = low variance = narrow curve. Low precision = high variance = wide curve.

6.3 Why the distinction matters

Here is the part that surprises people: **you can be precise and inaccurate at the same time.**



Accuracy and precision on a distribution. The reference value is the known true value.

Imagine a sensor that always reads exactly 5 N too high. Every reading will agree beautifully with every other reading — excellent precision — and every reading will be wrong by the same 5 N — terrible accuracy. Averaging more readings will *not* help. You will just get a very precise, very confident, wrong answer.

This is the most dangerous failure mode in estimation, and it matters directly for our race car:

- **Imprecision (random noise)** is what the Kalman filter is designed to fight. Averaging and filtering genuinely remove it.
- **Inaccuracy (bias)** is what the Kalman filter *cannot* fix, because the filter assumes the noise averages to zero. A biased sensor breaks that assumption, and the filter will confidently converge on the wrong answer.

The practical consequence: **calibrate your sensors before you filter them.**

Filtering is not a substitute for calibration. If your IMU has a gyroscope bias, no amount of clever filtering will save you — you must measure and subtract that bias first. (Systems that need to handle this properly estimate the bias as part of the state vector, so the filter can track and remove it.)

7. Systems, states, and the state vector

7.1 System and state

A **system** is whatever we have chosen to study. It can be a building, a person, a chemical process, a car, an aircraft — anything we want to measure things about.

A **state** is a quantity of that system that we have chosen to keep track of. For a person, that might be their height or their weight. For a car, it might be its velocity or its position.

More precisely: **the state is the minimum set of numbers you need to know in order to predict the system's future**, given what you do to it from now on. If you know a car's position, heading and speed, you can predict where it will be in one second. You do not need to know its colour. Colour is not part of the state.

7.2 The state vector

Usually one number is not enough, so we stack all the states we care about into a column of numbers called the **state vector**, written x .

For our race car, the pose (position and orientation) needs three numbers:

$$x = \begin{pmatrix} x \\ y \\ \psi \end{pmatrix}$$

where:

- x — position along the map's x-axis, in metres
- y — position along the map's y-axis, in metres
- ψ — the **yaw** angle (Greek letter "psi"), in radians: which way the car is pointing, measured anticlockwise from the x-axis

Three numbers, because the car drives on a flat surface. A drone would need six (three positions and three rotation angles). This number is the system's **degrees of freedom**.

7.3 The state vector in SLAM: it grows

Here is what makes SLAM different from plain localization, and it is the single most important structural idea in this project.

In SLAM we do not just estimate the car's pose. **We put the landmarks in the state vector too.** Each cone contributes its own x and y position, and the state vector grows every time we discover a new cone:

$$x = \begin{pmatrix} x_{\text{car}} & y_{\text{car}} & \psi_{\text{car}} & \div & m_{1x} & m_{1y} & m_{2x} & m_{2y} & \cdots \end{pmatrix}^T$$

where the first three entries are **the car's pose** and every pair after the divider is **one cone's position**. Written as a column it looks like this:

$$x = \begin{bmatrix} x_{\text{car}} \\ y_{\text{car}} \\ \psi_{\text{car}} \\ m_{1x} \\ m_{1y} \\ m_{2x} \\ m_{2y} \\ \vdots \end{bmatrix} \begin{array}{l} \leftarrow \text{car } x \\ \leftarrow \text{car } y \\ \leftarrow \text{car yaw} \\ \leftarrow \text{cone 1 } x \\ \leftarrow \text{cone 1 } y \\ \leftarrow \text{cone 2 } x \\ \leftarrow \text{cone 2 } y \\ \textcolor{red}{\vdots} \end{array}$$

So with 20 cones discovered, the state vector has $3 + 20 \times 2 = 43$ rows in it. At the start of the run it has just 3, because we have not seen any cones yet.

This is exactly what the code does. In `final_ekf.py` you will find:

```
STATE_SIZE = 3    # State size [x, y, yaw]
LM_SIZE     = 2    # Landmark state size [x, y]
```

and the helper that counts how many cones are currently in the state:

```
def calc_n_LM(x):
    n = int((len(x) - STATE_SIZE) / LM_SIZE)
    return n
```

which reads: *take the total length of the state, subtract the 3 rows belonging to the car, and divide what is left by 2, because each cone uses 2 rows.*

Why put the cones in the state at all? Because it lets the filter improve its estimate of the cones *and* the car together. If we see a cone we have seen before, that observation tells us something about where the car is — and simultaneously refines where we think that cone is. The two estimates are linked, and the mathematics of Chapter 9 (correlation) is what carries that link.

8. Sensor noise and process noise

Every filter needs to know about two completely different kinds of error. Getting these two straight is essential, because they enter the equations in different places and doing them the wrong way round will make your filter behave strangely.

8.1 Sensor noise (measurement noise)

Sensor noise is the error in the act of measuring. The world is what it is; our instrument reports it imperfectly.

Causes include uncalibrated or faulty sensors, thermal noise in electronics, vibration, limited resolution, and interference. As we established in Chapter 2, nothing measures perfectly.

We characterise sensor noise by taking many readings of the same fixed quantity and calculating the variance of those readings — exactly what we did with the punch. **The variance of a sensor's readings is the sensor noise.** It usually gets the symbol R (or Q in some textbooks — the two are annoyingly swapped between authors, so always check which convention a source is using).

The important property: sensor noise does not accumulate. Each reading is wrong independently of the last one. If your lidar is off by +3 cm this instant, that says nothing about whether it will be off by +3 cm or −2 cm in the next instant.

8.2 Process noise

Process noise is the error in our *model of how the world evolves*. Here the instrument is fine — reality simply did something our model did not account for.

Here is an example. We want to calculate the force of a car hitting a brick wall using $F = m \times a$. The car's mass is 800 kg and its acceleration is 50 m/s², so:

$$F = 800 \times 50 = 40,000 \text{ N}$$

But the measured force at impact was **25,000 N**. Why the enormous gap?

Could it be sensor noise? Possibly a little — but a 37% error is far too large to blame on the sensor. Something else happened. The most likely explanation is that the driver braked, and the car slowed before impact. If we work backwards from the measurement:

$$a = \frac{25,000}{800} = 31.25 \text{ m/s}^2$$

That is a perfectly plausible acceleration for a car that has shed some speed. Our sensor was fine. Our **model** was wrong, because it assumed constant acceleration and the real car did something else.

That gap between what our model predicts and what reality does is **process noise**, usually written Q (and confusingly written Cx in our code).

Other examples of process noise: - A car slowing down, speeding up or slipping when our model assumed constant velocity - A gust of wind pushing a drone sideways - Wheels slipping on a wet surface so odometry over-reports distance - A pilot making a manoeuvre a tracking system did not anticipate

8.3 The difference in one line

Sensor noise = the measurement was wrong. Process noise = the world did something the model did not predict.

And the key behavioural difference: **process noise accumulates, sensor noise does not**. Every second we run our motion model forward without a measurement, process noise piles up and our uncertainty grows. This is exactly the drift from §1.2. Sensor noise stays the same size no matter how long we run.

That is why the filter needs measurements. The prediction step lets uncertainty grow because of process noise; the update step pulls it back down using measurements. The whole filter is a tug of war between these two.

8.4 How this looks in our code

In `final_ekf.py`, the process noise for the car's pose is:

```
Cx = np.diag([0.25, 0.25, np.deg2rad(10.0)]) ** 2
```

Notice this is written as standard deviations that are then **squared** to give variances — which is a nice, readable way to write it, because standard deviations are the ones you can reason about physically. Reading it out:

- 0.25 m of uncertainty in x per prediction step $\rightarrow$ variance 0.0625 m^2
- 0.25 m of uncertainty in y per prediction step $\rightarrow$ variance 0.0625 m^2
- $10^\circ = 0.1745 \text{ rad}$ of uncertainty in yaw $\rightarrow$ variance 0.03046 rad^2

So this line is a statement about the car: *“each time I predict forward, I might be off by about 25 cm in position and about 10 degrees in heading.”* That is the engineer's judgement about how much to distrust the motion model, and it is one of the two main tuning knobs of the whole filter.

Note that the values are written as standard deviations and then squared. That is worth doing deliberately: standard deviations are the ones you can reason about physically, so writing them that way keeps the line readable.

9. Covariance: uncertainty in more than one dimension

Everything so far has been about a single number with a single variance. Our state vector has at least three numbers in it, so we need to extend the idea. This chapter is new — it was missing from the original document, and its absence made several later chapters much harder to follow than they needed to be.

9.1 The problem with treating dimensions separately

Suppose our car's position estimate is “ $x=10\text{ m}\pm0.5\text{ m}$, $y=4\text{ m}\pm0.5\text{ m}$ ”. You might picture a circle of uncertainty around the point $(10,4)$.

But now suppose the car has been driving in a straight line along a diagonal. We know quite accurately *how far along that diagonal* it has gone, but we are much less sure about sideways drift. The region where the car might really be is not a circle — it is a long thin **ellipse**, tilted along the direction of travel.

To describe a tilted ellipse you need more than two variances. You need to know how the errors in x and y **relate to each other**. That relationship is called covariance.

9.2 The covariance matrix

We collect everything into a square matrix, written P . For our three-number pose:

$$P = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{x\psi} \\ \sigma_{yx} & \sigma_{yy} & \sigma_{y\psi} \\ \sigma_{\psi x} & \sigma_{\psi y} & \sigma_{\psi\psi} \end{bmatrix}$$

Reading this matrix:

- **The diagonal entries** (σ_{xx} , σ_{yy} , $\sigma_{\psi\psi}$) are the plain variances of each state on its own. σ_{xx} is “how uncertain am I about x ”. These are always positive.
- **The off-diagonal entries** are the **covariances** — how errors in one state relate to errors in another. σ_{xy} answers: “when my x estimate is too big, does my y estimate also tend to be too big?” These can be positive, negative, or zero.

The matrix is always **symmetric** ($\sigma_{xy}=\sigma_{yx}$), because “how x relates to y ” is the same statement as “how y relates to x ”.

What the off-diagonal values mean:

Value	Meaning
Positive	The two errors move together. If x is overestimated, y probably is too.
Negative	The two errors move oppositely. If x is overestimated, y is probably underestimated.

Value	Meaning
Zero	The two errors are unrelated. Knowing one tells you nothing about the other.

9.3 Why this matters enormously for SLAM

Here is the part that makes SLAM work, and it is worth reading twice.

In SLAM, the covariance matrix covers the *whole* state vector — car pose **and** every cone. With 20 cones that is a 43×43 matrix. The off-diagonal blocks connecting the car to the cones, and the cones to each other, are where the magic lives.

Think about what happens physically. We see a cone from a certain car position, so we place that cone on the map. If our car position was slightly wrong, our cone position is wrong in *exactly the same direction*. The two errors are correlated, and the covariance matrix records that correlation.

Now, much later, we drive round the track and see that same cone again. The measurement corrects our estimate of the cone — and because the matrix knows the cone's error is linked to the car's error, **the correction propagates back and fixes the car's position too**. And because that cone is also correlated with every other cone we placed from nearby positions, the whole map shifts into better alignment.

That is loop closure. It is the single most valuable behaviour in SLAM, and it is delivered entirely by the off-diagonal blocks of P . If you threw the off-diagonals away and kept only the variances, the filter would still run, but it would never improve the map this way.

9.4 Visualising it

The covariance matrix is best pictured as an **uncertainty ellipse**:

- **Large diagonal, zero off-diagonal** → a big circle. Uncertain in every direction equally.
- **Small diagonal, zero off-diagonal** → a small circle. Confident in every direction.
- **Non-zero off-diagonal** → a tilted ellipse. Confident along one direction, uncertain along another.

The tilted ellipse is the interesting case, and it is what a real robot's uncertainty almost always looks like — a car driving forward accumulates error along its direction of travel differently from sideways.

9.5 A three-word summary

If you remember one sentence from this chapter:

The mean says where we think we are. The covariance says how wrong we might be, and in which directions.

Every equation in the rest of this document manipulates exactly these two objects. The state vector x , and the covariance matrix P . That is the whole filter.

Part II — The filters

10. The core idea: predict and update

Before any equations, I want to plant the central idea, because every filter in this document is a variation on it.

10.1 Two sources of information

At any moment we have two independent ways of knowing where the car is.

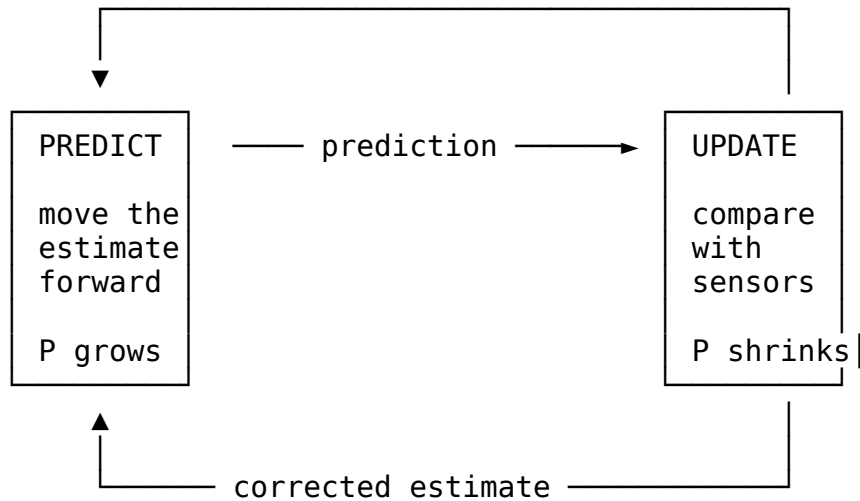
One: we know where we were, and what we have been doing. We were at position 10 m, we have been driving at 2 m/s, and 0.1 seconds have passed, so we are probably at 10.2 m now. Call this the **prediction**. Its weakness is that it is built on assumptions, so it gets worse the longer we go without checking.

Two: we can look at the world. We can see a cone and measure how far away it is. Call this the **measurement**. Its weakness is that sensors are noisy, so any single reading is unreliable.

Neither is trustworthy alone. But they fail in *different ways*, and that is what makes them combinable. The prediction is smooth but drifts. The measurement is jumpy but does not drift. A filter that combines them gets a result that is both smooth and drift-free — better than either input.

10.2 The loop

The filter runs this cycle forever:



Predict — move the estimate forward in time using a model of how the car moves. Uncertainty **grows**, because the model is imperfect (process noise).

Update — take a measurement, see how much it disagrees with the prediction, and nudge the estimate towards the measurement by an amount that reflects which one you trust more. Uncertainty **shrinks**, because you have gained information.

That is it. Prediction inflates uncertainty; measurement deflates it. Everything else is bookkeeping.

10.3 Where this comes from

This structure is not invented ad hoc. It falls out of **Bayes' rule**, the fundamental law for updating a belief in light of new evidence:

$$p(x_i | z) \propto p(z_i | x_i) \times p(x_i)$$

In words: *your new belief is your old belief, reweighted by how well each possibility explains what you just measured.* A general algorithm that applies this recursively is called a **Bayes filter**.

The Bayes filter is completely general but usually impossible to compute, because tracking an arbitrary probability shape over time is intractable. **The Kalman filter is what you get when you assume every distribution involved is Gaussian and every model is linear.** Under those two assumptions, all the integrals collapse into simple matrix algebra, and the whole thing becomes something a computer can run thousands of times per second.

That is the trade the Kalman filter makes: give up generality, gain enormous speed. Chapters 13–16 are about what happens when the assumptions do not hold, and how the EKF patches them up.

11. The Kalman filter

11.1 What it is

The **Kalman filter**, published by Rudolf Kálmán in 1960, is an algorithm that estimates the state of a linear system from noisy measurements. It is the workhorse of estimation: it flew on Apollo’s navigation computer, and it runs today in essentially every GPS receiver, aircraft, and robot in existence.

It takes in:

- a **model** of how the system moves, and how much to distrust it (process noise Q)
- **measurements**, and how much to distrust them (sensor noise R)

and produces the estimate with the **smallest possible variance** — provably, if the system really is linear and the noise really is Gaussian. Not “a good estimate”, but the mathematically optimal one. That is a strong guarantee and it is why the algorithm is so widely trusted.

11.2 The two-step structure

The filter keeps exactly two things:

- $\hat{x}$ — the state estimate (our best guess). The hat means “estimated”.
- P — the covariance matrix (how wrong we might be).

And runs the two steps from Chapter 10.

Step 1: Predict

$$\hat{x}_k^- = F \hat{x}_{k-1} + B u_k$$

$$P_k^- = F P_{k-1} F^T + Q$$

Term by term:

- F — the **state transition matrix**. Encodes the physics: “given where I was, where will I be?”

- u_k — the **control input**. What we commanded the system to do (throttle, steering).
- Q — the **process noise**. How much we distrust our own model.
- The superscript $-$ means “predicted, before seeing the measurement”. Often called the *prior*.

The first line moves our guess forward. The second line moves our uncertainty forward — and notice that Q is **added**, so P always grows during prediction. Uncertainty can only increase when you are guessing.

The $F P F^T$ sandwich is how you push a covariance through a linear transformation. Intuitively: if you stretch the state by F , the uncertainty ellipse stretches too, and because variance involves squares, F appears twice — once on each side.

Step 2: Update

$$\tilde{y}_k = z_k - H \hat{x}_k^- \text{ (innovation)}$$

$$S_k = H P_k^- H^T + R \text{ (innovation covariance)}$$

$$K_k = P_k^- H^T S_k^{-1} \text{ (Kalman gain)}$$

$$\hat{x}_k = \hat{x}_k^- + K_k \tilde{y}_k \text{ (corrected state)}$$

$$P_k = (I - K_k H) P_k^- \text{ (corrected covariance)}$$

This looks like a lot, so let us take it one line at a time. It is genuinely simpler than it looks.

11.3 The update, line by line

The measurement matrix H . Our state might contain things we cannot measure directly. H converts a state into the measurement we *would expect to see* if that state were true. If the state is [position, velocity] and our sensor only reads position, then $H = [1 \ 0]$: take the position, ignore the velocity. H translates from “state language” into “sensor language”.

Line 1 — the innovation $\tilde{y}$. This is the most intuitive quantity in the whole filter:

$$\tilde{y} = \underset{\text{measurement}}{z} - H \underset{\text{predicted state}}{\hat{x}^-}$$

It is the **surprise**. How wrong was my prediction? If the innovation is zero, my prediction was perfect and nothing needs to change. If it is large, my prediction was badly off. Everything the filter learns, it learns from the innovation.

Line 2 — the innovation covariance S . How surprised should I have expected to be? Two things make surprise unsurprising: my own uncertainty about the state ($H P^- H^T$, my state uncertainty expressed in sensor units) and the sensor's own noise (R). Add them and you get the expected size of the innovation.

Line 3 — the Kalman gain K . This is the heart of the filter. It decides **how much to trust the measurement versus the prediction**. Look at its structure, ignoring matrices for a moment and imagining plain numbers:

$$K = \frac{P}{P + R}$$

Now read off the two extremes:

- **My prediction is terrible** (P huge compared to R): $K \rightarrow 1$. Trust the measurement almost completely — throw away the prediction and jump to what the sensor says.
- **My sensor is terrible** (R huge compared to P): $K \rightarrow 0$. Ignore the measurement and stick with the prediction.

K slides between 0 and 1 depending on which source currently deserves more trust. And it does this *automatically, every cycle*, recomputed from the current uncertainties. That is the elegance of the algorithm: nobody has to decide how much to trust what — the mathematics works it out from the covariances.

This is Chapter 5's rule made quantitative: **trust the narrower Gaussian more**.

Line 4 — correct the state. Take the prediction and nudge it towards the measurement, by an amount equal to the surprise scaled by how much we trust it:

$$\text{new estimate} = \text{prediction} + \text{gain} \times \text{surprise}$$

Line 5 — correct the covariance. Since K is between 0 and 1, $(I - KH)$ is less than 1, so P always **shrinks**. Getting a measurement always reduces uncertainty, and the more you trusted the measurement, the more it shrinks.

11.4 A complete worked example with real numbers

Abstract equations are hard to believe, so let us run the actual filter on the punch example from Chapter 2 and watch it work.

Setup. We want to estimate the true force. Our physics model $F=ma$ said **550 N**, but we do not trust it much — call it ± 30 N, so $P_0=30^2=900$. Our sensors have variance $R=171.973$ (calculated in Chapter 3). The force is not changing, so the prediction step is just “the force stays the same”, with a small process noise $Q=1$ to allow for slight drift.

Now we feed in the five readings one at a time:

Step	Measure ment z	Gain K	Innovatio n $\tilde{y}$	New estimate $\hat{x}$	New P	New σ
start	—	—	—	550.00	900.00	30.00
1	530.7	0.840	−19.30	533.79	144.41	12.02
2	531.2	0.458	−2.59	532.61	78.79	8.88
3	530.1	0.317	−2.51	531.81	54.50	7.38
4	535.4	0.244	+3.59	532.69	41.96	6.48
5	560.8	0.200	+28.11	538.31	34.37	5.86

Read what happened, because every column tells a story.

The gain falls steadily — 0.84, 0.46, 0.32, 0.24, 0.20. At the start the filter knew almost nothing ($P=900$, far bigger than $R=172$), so it threw away its prediction and leapt to the first measurement. As it accumulated information, P shrank, and the filter grew progressively more sceptical of new measurements. **The filter learned how much to trust itself.** Nobody programmed that behaviour; it falls out of the gain formula.

Watch step 5. The outlier of 560.8 N arrives with a huge innovation of +28.1 N. Early on, that would have dragged the estimate 23 N upwards. But by step 5 the gain is only 0.20, so the estimate moves just $0.20 \times 28.1 = 5.6$ N. **The filter has become robust to outliers all by itself**, simply because it is now confident.

The uncertainty falls monotonically — $30.0 \rightarrow 12.0 \rightarrow 8.9 \rightarrow 7.4 \rightarrow 6.5 \rightarrow 5.9$ N. Every measurement makes us more certain. Note the final σ of **5.86 N** against the raw sensor spread of **13.11 N**: the filter has more than halved the uncertainty.

That is real information gained from combining measurements, and it is the payoff for all this machinery.

And the estimate itself: $550 \rightarrow 533.8 \rightarrow 532.6 \rightarrow 531.8 \rightarrow 532.7 \rightarrow 538.3$ N. It moved decisively away from the over-confident model value of 550, settled near the tight cluster of readings around 531, and then made a modest, appropriately sceptical adjustment upwards when the outlier arrived.

11.5 The assumptions

The optimality guarantee comes with strings attached. The Kalman filter is optimal **only if**:

1. The system is **linear** — the next state is a linear function of the current state.
2. The noise is **Gaussian**.
3. The noise is **zero-mean** — no bias (this is §6.3 again).
4. Process and measurement noise are **uncorrelated** with each other.

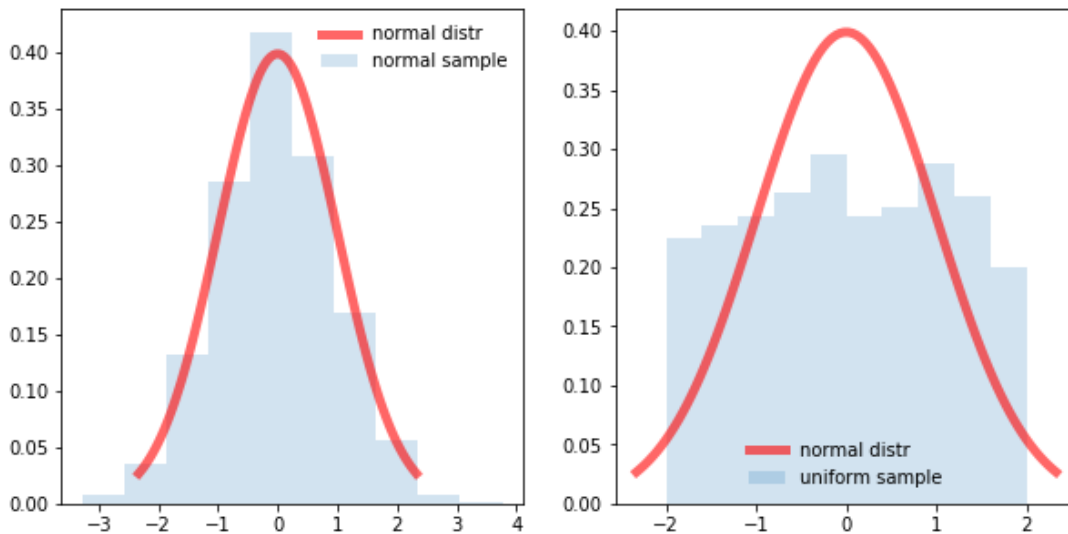
Assumption 1 is the one that breaks on a real car, and breaking it is what the entire rest of Part II is about.

Further reading. The single best free resource on the Kalman filter is Alex Becker's kalmanfilter.net, which builds up from a one-dimensional example with full numerical tables. If you want more depth with runnable code, Roger Labbe's *Kalman and Bayesian Filters in Python* is outstanding and free. Both are in the References.

12. Fusing two Gaussians

12.1 The question

We have two independent estimates of the same quantity. Say our model predicts the force is 550 N with variance 900, and our sensors say 537.64 N with variance 171.97. How do we combine them into one better estimate?



Two Gaussians: a wide prediction and a narrower measurement

12.2 We multiply, we do not add

It is worth being careful here, because the obvious guess is wrong. We do **not** add the two distributions. If you add two independent random variables their variances *add*, and the result is **more** uncertain than either input — the opposite of what a filter is for.

The correct operation is **multiplication of the two probability density functions**. This is Bayes' rule in action: one Gaussian is the prior belief, the other is the likelihood of the measurement, and Bayes says multiply them. Multiplying is what makes the variance *shrink*.

The intuition for why multiplication is right: a value is plausible only if *both* sources say it is plausible. Where one curve is near zero, the product is near zero, regardless of what the other says. Multiplication is logical AND for probabilities, and requiring both sources to agree is exactly what narrows things down.

12.3 The formulas

Multiply two Gaussians (μ_1, σ_1^2) and (μ_2, σ_2^2) and — remarkably — you get another Gaussian, with:

$$\sigma_{\text{fused}}^2 = \frac{1}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}} \quad \mu_{\text{fused}} = \sigma_{\text{fused}}^2 \left(\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2} \right)$$

The quantity $1/\sigma^2$ appearing everywhere has a name: **precision** (also called *information*). The formulas become memorable when you say them in those terms:

Precisions add. The fused mean is the average of the two means, weighted by their precisions.

“Precisions add” is the key line. Since precisions add and precision is one-over-variance, the fused variance is *always smaller than either input variance*. Combining information always helps, even if one source is poor.

12.4 Our numbers

$\mu_1=550$, $\sigma_1^2=900$ (the model). $\mu_2=537.64$, $\sigma_2^2=171.97$ (the sensors).

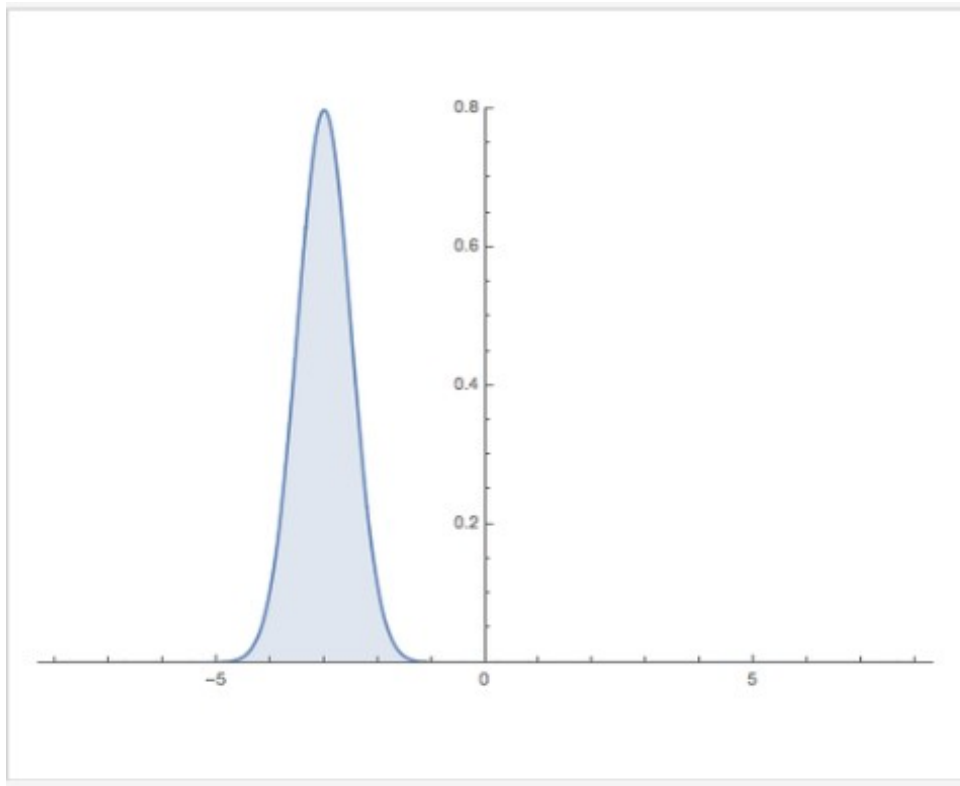
$$\sigma_{\text{fused}}^2 = \frac{1}{\frac{1}{900} + \frac{1}{171.97}} = \frac{1}{0.001111 + 0.005815} = \frac{1}{0.006926} = 144.38$$

$$\mu_{\text{fused}} = 144.38 \times \left(\frac{550}{900} + \frac{537.64}{171.97} \right) = 144.38 \times (0.6111 + 3.1264) = 539.62$$

Result: 539.62 N with variance 144.38, so $\sigma=12.02$ N.

Look at what happened:

- The fused variance (144.38) is **smaller than either input** (900 and 171.97). We gained information.
- The fused mean (539.62) sits much closer to the sensor value (537.64) than the model value (550), because the sensor was over five times more precise. **It was pulled towards the more trustworthy source, automatically.**



The result of fusing the two distributions: narrower than either input

Notice the variance became much smaller — meaning our error range dropped and our estimate became more reliable. That is exactly what we want a filter to do.

12.5 This is the Kalman update

Here is the satisfying part. Define $K = \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2} = \frac{900}{900 + 171.97} = 0.8396$. Then:

$$\mu_{\text{fused}} = \mu_1 + K(\mu_2 - \mu_1) = 550 + 0.8396(537.64 - 550) = 539.62 \checkmark$$

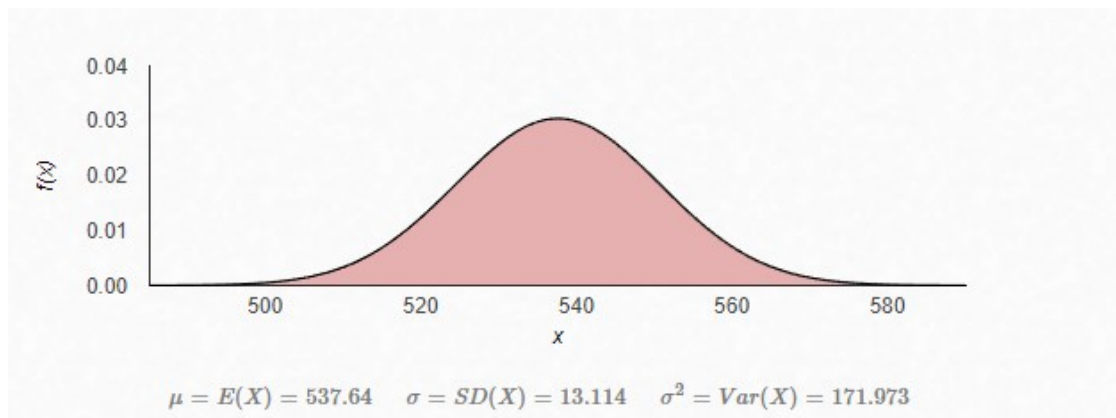
$$\sigma_{\text{fused}}^2 = (1 - K)\sigma_1^2 = (1 - 0.8396) \times 900 = 144.38 \checkmark$$

Identical answers. And those two lines are exactly lines 4 and 5 of the Kalman update from §11.2, with $H = 1$.

So the Kalman filter update step is nothing more than multiplying two Gaussians. All the matrices in Chapter 11 are there to handle many dimensions at once; the underlying operation is this simple. If you understood this section, you understand the Kalman filter.

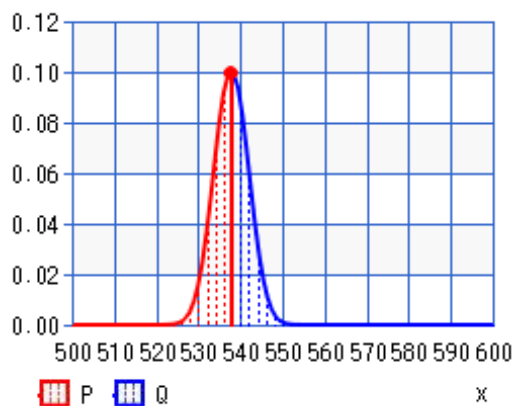
12.6 Our force example, before and after

Before — the raw sensor distribution, $\sigma = 13.11$ N:



Before filtering

After — the filtered distribution from §11.4, $\sigma = 5.86$ N:



After filtering

The covariance is now dramatically less than our initial estimate. Same sensors, same data — better answer, purely from combining information properly.

13. Why the Kalman filter breaks

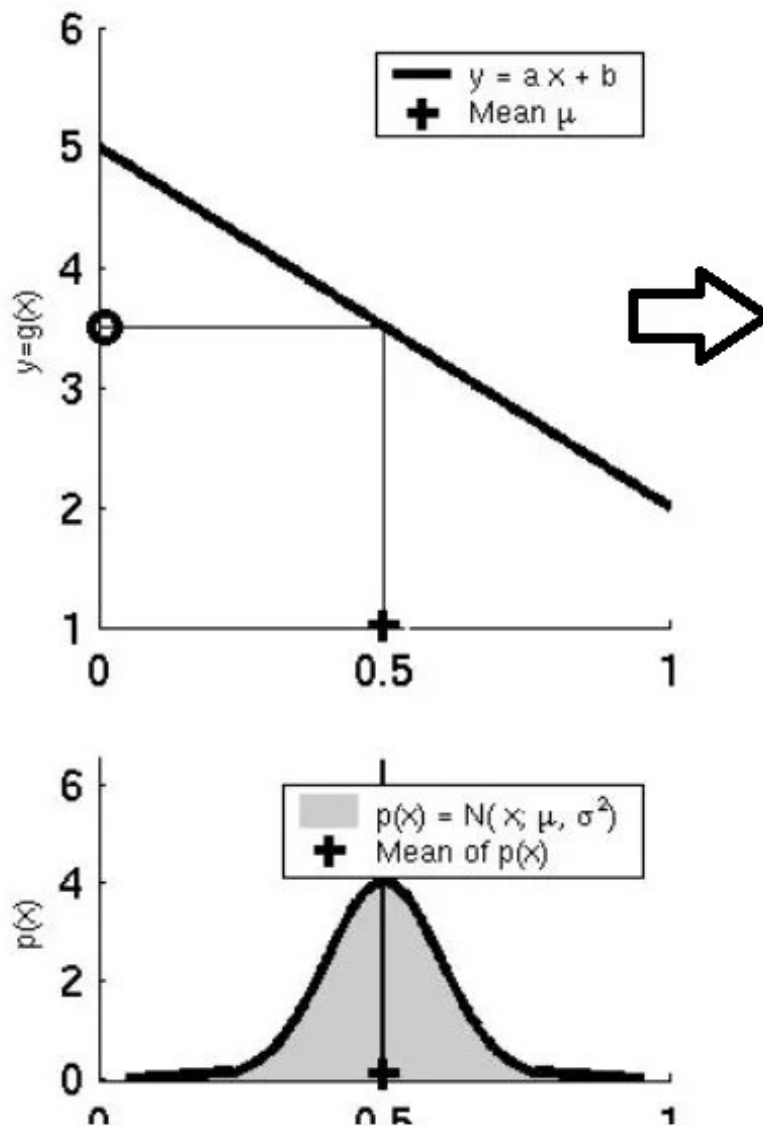
We now have a filter that is provably optimal. Unfortunately we cannot use it on a car. Here is why.

13.1 Linear functions and what they preserve

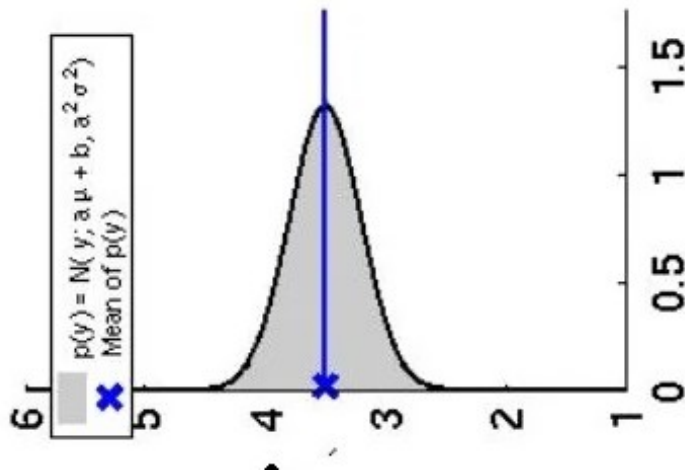
A function is **linear** if its graph is a straight line: $y = ax + b$. Doubling the input doubles the change in output; there are no squares, no sines, no divisions by variables.

Linear functions have a property that the Kalman filter depends on completely:

Push a Gaussian through a linear function and you get another Gaussian.



A Gaussian passed through a linear function comes out Gaussian

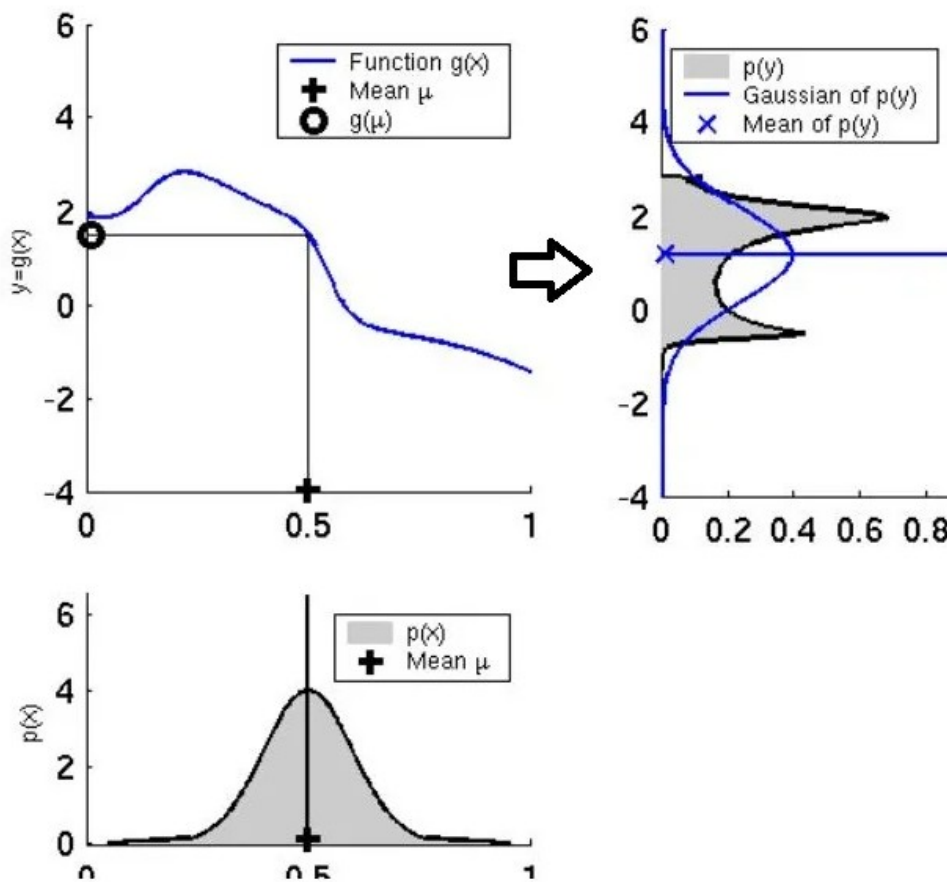


The output is still a well-formed bell curve

The mean shifts and the width changes, but the shape survives. That is what lets the filter keep working with just (μ, σ^2) forever — after any number of linear steps, two numbers still describe everything.

13.2 Non-linear functions destroy this

Now push a Gaussian through a non-linear function — a sine, a square root, an arctangent:



A Gaussian passed through a non-linear function comes out distorted and non-Gaussian

The output is **not a Gaussian**. It is skewed, possibly lopsided, possibly with several humps. And now we are stuck: computing a “mean and variance” for that shape does not describe it any more. The two numbers we are carrying no longer capture the truth, and the Kalman filter’s guarantees evaporate.

To be precise about what has gone wrong: you can always *compute* a mean and a variance for any distribution. The problem is that they no longer *fully describe* it, because the distribution is no longer Gaussian and a non-Gaussian shape needs more than two numbers to pin down. It is that information loss which breaks the filter, not any impossibility in the arithmetic.

13.3 Why our car is unavoidably non-linear

Two places, and neither can be avoided.

The motion model. When the car moves forward by distance d at heading ψ , its new position is:

$$x_{\text{new}} = x + d \cos(\psi) \quad y_{\text{new}} = y + d \sin(\psi)$$

There is cos and sin of a state variable. Sine and cosine are about as non-linear as functions get. Any vehicle that can turn has this problem — it is fundamental to driving in two dimensions.

The measurement model. The lidar sees a cone and reports how far away it is and at what angle. If the cone is at (m_x, m_y) and the car is at (x, y, ψ) :

$$\text{range} = \sqrt{(m_x - x)^2 + (m_y - y)^2}$$

$$\text{bearing} = \arctan\left(\frac{m_y - y}{m_x - x}\right) - \psi$$

A square root of a sum of squares, and an arctangent of a ratio. Both thoroughly non-linear.

Put another way: in most cases the system is looking in one direction and taking a measurement in another. That involves angles, sines and cosines, which are non-linear and lead to problems.

13.4 The way out

We cannot make the car linear. So we cheat, in a specific and controlled way:

If the function is not linear, replace it with a straight line that is a good approximation *near where we currently are*.

This is called **linearization**. It is not exact and it introduces error. But if our uncertainty is small — if we are only considering a narrow range of possible positions — a curve looks very nearly straight over that narrow range, and the error is tolerable.

The mathematical tool for finding that best-fitting straight line is the **Taylor series**, and the version of it that works for multi-dimensional systems is the **Jacobian matrix**. Those are the next two chapters, and once we have them we can build the EKF.

14. Taylor series: turning curves into straight lines

The Taylor series deserves its own chapter, because it is the entire mathematical justification for the EKF.

14.1 The idea

Pick any smooth curve and zoom in far enough on any point of it. It looks straight. The Earth is a sphere, but the floor of your room looks flat, because you are only seeing a tiny patch of it.

The **Taylor series** makes this precise. It says: *near a chosen point, any smooth function can be rebuilt from its value and its derivatives at that point.*

$$f(x) \approx \underset{i}{f(a)} + \underset{i}{f'(a)}(x - a) + \frac{\underset{i}{f''(a)}}{2!}(x - a)^2 + \frac{\underset{i}{f'''(a)}}{3!}(x - a)^3 + \dots$$

where a is the point we are expanding around and f' , f'' are the first and second derivatives.

Read the terms as increasingly good answers to “what does the function do near a ?”:

- **Term 1** ($f(a)$): “it is about this tall.” A flat line.
- **Term 2** ($f'(a)(x - a)$): “and it is heading in this direction.” A sloped line — the tangent.
- **Term 3**: “and it curves this way.” A parabola.
- and so on, each term correcting the last.

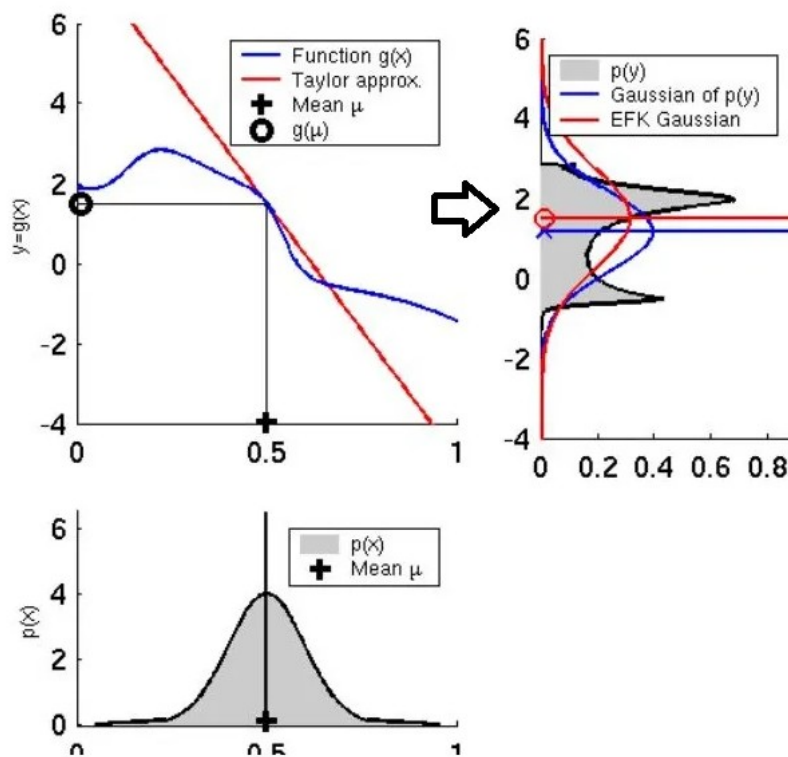
It is worth being clear about what the series *is*: not the derivatives themselves, but an **infinite sum of terms built from those derivatives**, each weighted by $(x - a)^n / n!$. The weights are what make the approximation improve as you add terms.

14.2 The EKF keeps only the first two terms

Here is the punchline. The EKF throws away everything from the curvature term onwards:

$$f(x) \approx f(a) + f'(a)(x - a)$$

This is called the **first-order Taylor expansion**, and geometrically it is just **the tangent line at a** :



Linearizing a curve by taking the tangent at the mean

We take the mean of our Gaussian, put it on the non-linear curve, draw the tangent line there, and pretend the function is that straight line. Now the function is linear, so we can use the Kalman filter equations.

The name “Extended Kalman Filter” means precisely this: a Kalman filter extended to non-linear systems by replacing every non-linear function with its tangent line at the current best estimate.

14.3 How good is the approximation? Real numbers.

Let us actually check, because “it is approximately linear” should never be taken on faith.

Take $f(x) = \sin(x)$ expanded around $a=0$. Since $\sin(0)=0$ and $\cos(0)=1$, the first-order approximation is simply $f(x) \approx x$.

x (rad)	true $\sin(x)$	approximation x	error
0.1 (5.7°)	0.099833	0.100000	0.000167
0.5 (28.6°)	0.479426	0.500000	0.020574

x (rad)	true $\sin(x)$	approximation	error
		x	
1.0 (57.3°)	0.841471	1.000000	0.158529

And $f(x) = \cos(x)$ around $a=0$, where the approximation is the flat line $f(x) \approx 1$:

x (rad)	true $\cos(x)$	approximation	error
0.1	0.995004	1.0	0.004996
0.5	0.877583	1.0	0.122417

Read the error column carefully, because it contains the central engineering lesson of the EKF.

At 0.1 radians the error is 0.000167 — utterly negligible. At 0.5 radians it is 0.021, over a hundred times worse. At 1.0 radians it is 0.159, which is a 19% error and completely unacceptable.

The error grows roughly with the **square** of the distance from the expansion point. So:

The EKF works well when uncertainty is small and badly when uncertainty is large.

This single sentence explains most EKF failures in practice:

- **Run the filter fast.** A short Δt means the car moves a small distance between updates, so we never linearize far from the expansion point. This is why a control loop at 50 Hz behaves far better than one at 5 Hz, quite apart from responsiveness.
- **Keep uncertainty low.** If P becomes large, the Gaussian is wide, we are linearizing over a wide range, and the approximation degrades. Worse, a bad linearization produces a bad update, which can make P wrong, which makes the next linearization worse. **The EKF can diverge** — spiral into confident nonsense — and this feedback loop is how.
- **Sharp turns are the dangerous moments.** Driving straight, the heading barely changes and $\sin/\cos$ are nearly linear over the relevant range. In a hairpin, the heading swings fast and the linearization is being asked to cover a much wider angle. This is exactly where a Formula Student EKF is most likely to misbehave.

15. Jacobians: derivatives for systems with many variables

15.1 Why we need a new tool

Chapter 14 linearized $f(x)$ — one input, one output. Our car has three state variables and produces two measurement values. We need the derivative of a function that takes several inputs and produces several outputs.

That object is the **Jacobian matrix**.

15.2 Partial derivatives

First, a smaller idea. If a function has several inputs, a **partial derivative** asks: *how much does the output change when I wiggle just one input and hold all the others still?* It is written with a curly ∂ instead of a d .

For $f(x, y) = x^2 + 3y$:

- $\frac{\partial f}{\partial x} = 2x$ — treat y as a constant, so the $3y$ term differentiates to zero.
- $\frac{\partial f}{\partial y} = 3$ — treat x as a constant, so the x^2 term differentiates to zero.

That is all a partial derivative is: ordinary differentiation, with every other variable frozen.

15.3 The Jacobian matrix

The **Jacobian** collects every partial derivative of every output with respect to every input into a grid:

$$\frac{\partial(x, y)}{\partial(u, v)} = \begin{vmatrix} \frac{\partial x}{\partial u} & \frac{\partial x}{\partial v} \\ \frac{\partial y}{\partial u} & \frac{\partial y}{\partial v} \end{vmatrix}$$

The Jacobian matrix as a grid of partial derivatives

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

The layout rule is worth saying explicitly because it is easy to get backwards:

Row i , column j = how much output i changes when input j changes.

One row per output. One column per input.

So a function with 2 outputs and 5 inputs has a 2×5 Jacobian. (This will matter in §19.9, where we build exactly that.)

15.4 What it means

The Jacobian is the multi-dimensional version of “the slope”. It is the best linear approximation of a non-linear function at a particular point.

Everything from Chapter 14 carries over directly. Where the one-dimensional Taylor expansion was:

$$f(x) \approx f(a) + f'(a)(x - a)$$

the multi-dimensional version is:

$$f(x) \approx f(a) + J(x - a)$$

Same equation. The Jacobian simply plays the role that $f'(a)$ played before.

One clarification, because it is a common source of confusion. Jacobians *do* appear in coordinate transformations, which is where most people first meet them in a multivariable calculus course. That is one application, but it is not what they are, and it is not what they do here. **A Jacobian is the matrix of first-order partial derivatives: the best linear approximation of a non-linear function at a point.** In the EKF it is used purely for linearization, not for changing coordinates.

15.5 A worked example: the exact model our lidar uses

Let us build the Jacobian of the range-and-bearing measurement, since we need it later anyway.

The function takes a relative position $(\Delta x, \Delta y)$ and returns range and bearing:

$$h(\Delta x, \Delta y) = \begin{bmatrix} r \\ \phi \end{bmatrix} = \begin{bmatrix} \sqrt{\Delta x^2 + \Delta y^2} \\ \arctan\left(\frac{\Delta y}{\Delta x}\right) \end{bmatrix}$$

Two inputs, two outputs, so the Jacobian is 2×2 . Working out the four partial derivatives (writing $q = \Delta x^2 + \Delta y^2$, so $r = \sqrt{q}$):

$$J = \begin{bmatrix} \frac{\partial r}{\partial \Delta x} & \frac{\partial r}{\partial \Delta y} \\ \frac{\partial \phi}{\partial \Delta x} & \frac{\partial \phi}{\partial \Delta y} \end{bmatrix} = \begin{bmatrix} \frac{\Delta x}{\sqrt{q}} & \frac{\Delta y}{\sqrt{q}} \\ -\frac{\Delta y}{q} & \frac{\Delta x}{q} \end{bmatrix}$$

Numbers. Suppose the cone is at $\Delta x = 3$, $\Delta y = 4$. Then $q = 9 + 16 = 25$ and $r = 5$:

$$J = \begin{bmatrix} 3/5 & 4/5 \\ -4/25 & 3/25 \end{bmatrix} = \begin{bmatrix} 0.60 & 0.80 \\ -0.16 & 0.12 \end{bmatrix}$$

Now let us check that it actually works. Move the cone slightly, by $\Delta x \rightarrow 3.1$ and $\Delta y \rightarrow 3.95$:

- **True values:** range $\sqrt{3.1^2 + 3.95^2} = 5.02116$, bearing 0.90536 rad
- **Linear approximation:** $\begin{bmatrix} 5 \\ 0.92730 \end{bmatrix} + \begin{bmatrix} 0.60 & 0.80 \\ -0.16 & 0.12 \end{bmatrix} \begin{bmatrix} 0.10 \\ -0.05 \end{bmatrix} = \begin{bmatrix} 5.02000 \\ 0.90530 \end{bmatrix}$
- **Error:** 0.0012 in range, 0.00006 rad in bearing

The linear approximation is accurate to about a millimetre over a 10 cm move.

This is why the EKF works. For small movements between filter cycles, the tangent-line approximation is genuinely excellent — and Chapter 14 already told us why it would be, and warned us what happens when the movement is not small.

15.6 The two Jacobians in our filter

The EKF needs exactly two, and they replace exactly two matrices from the linear Kalman filter:

Jacobian	Symbol	Of what	Replaces	Built in code by
Motion Jacobian	G	the motion model	F	<code>jacob_motion()</code>
Measurement Jacobian	H	the	H	<code>jacobH()</code>

Jacobian	Symbol	Of what	Replaces	Built in code by
t Jacobian		measurement model		

That is the whole modification. Take the Kalman filter, and everywhere a model matrix appears in a *covariance* equation, put a Jacobian instead.

16. The EKF equations, line by line

We now have every piece. Let us assemble the filter.

16.1 The key insight

The EKF makes one change to the Kalman filter, and it is worth stating very precisely because it is easy to get wrong:

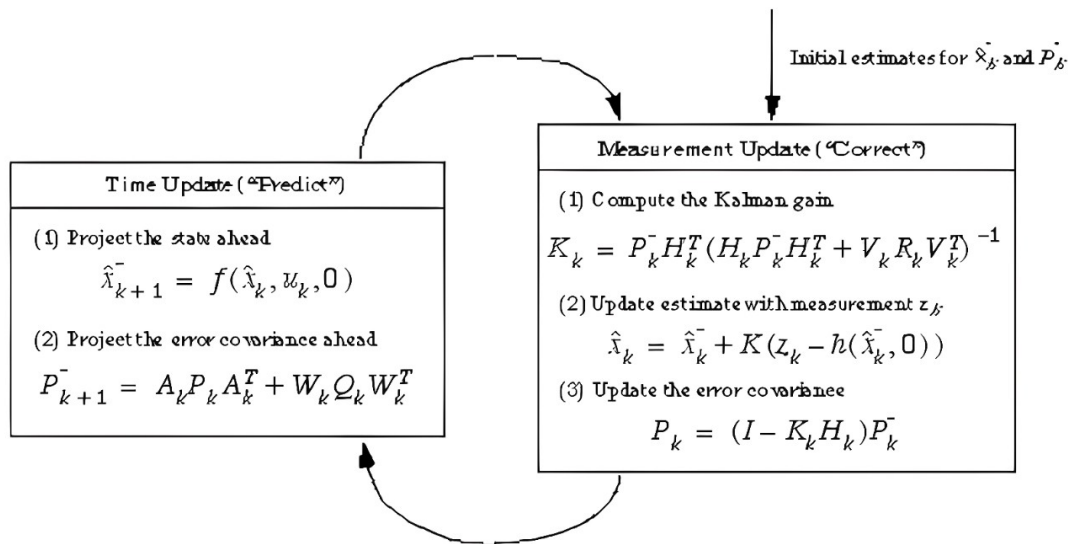
Use the full non-linear function to move the *state*. Use the Jacobian to move the *covariance*.

Why the split? Because they are doing different jobs.

- The **state** is a single point. We can push a single point through the exact non-linear function with no approximation at all — there is no reason to approximate here, so we do not.
- The **covariance** describes a whole cloud of uncertainty. There is no exact way to push a cloud through a non-linear function and get a Gaussian out. So here we have no choice: we approximate the function by its tangent, and push the covariance through that.

Approximate only where you must.

16.2 The equations



The EKF predict-update cycle

Predict

$\hat{x}_k^- = f(\hat{x}_{k-1}, u_k)$ — exact non-linear motion model

$P_k^- = G_k P_{k-1} G_k^T + Q$ — Jacobian G replaces F

Update

$\tilde{y}_k = z_k - h(\hat{x}_k^-)$ — exact non-linear measurement model

$$S_k = H_k P_k^- H_k^T + R$$

$$K_k = P_k^- H_k^T S_k^{-1}$$

$$\hat{x}_k = \hat{x}_k^- + K_k \tilde{y}_k$$

$$P_k = (I - K_k H_k) P_k^-$$

Compare against §11.2. The differences are exactly two: $F \hat{x}$ became $f(\hat{x})$, and $H \hat{x}^-$ became $h(\hat{x}^-)$; and in the covariance lines, F and H became the Jacobians G and H . **Everything else is identical.**

16.3 The subscript k matters

In the linear Kalman filter, F and H are constants — computed once, used forever.

In the EKF, G_k and H_k carry a subscript k because they must be **recomputed at every single time step**. The tangent line depends on where you draw it, and the car has moved since last time, so last cycle's tangent is the tangent at the wrong place.

This is the main extra computational cost of the EKF, and it is also the main place bugs hide. If your EKF misbehaves, the Jacobian is the first thing to check.

16.4 A summary table

	Kalman Filter	Extended Kalman Filter
System	Must be linear	Can be non-linear
State prediction	$F \hat{x}$	$f(\hat{x})$ — exact
Covariance prediction	$F P F^T + Q$	$G P G^T + Q$
F, H	Constant	Recomputed every step as Jacobians
Optimal?	Yes , provably	No — approximate
Can it diverge?	No (if assumptions hold)	Yes
Cost	Low	Moderate (Jacobians each step)

That “No” in the optimality row is important and honest. **The EKF is a heuristic.** A very good, extremely widely used heuristic that has flown spacecraft — but it carries no guarantee, and it can fail. Treat any EKF with respect and test it hard.

16.5 What people use when the EKF is not good enough

For completeness, since you will meet these names:

- **UKF (Unscented Kalman Filter)** — instead of linearizing, pick a small set of carefully chosen sample points (“sigma points”), push each one through the *exact* non-linear function, and rebuild a Gaussian from where they land. Similar cost to the EKF, usually more accurate, and no Jacobians to derive — which removes a whole class of bugs. If I rebuilt this project, I would try a UKF.
- **Particle filter** — represent the belief with thousands of weighted samples. Handles any distribution, including multi-modal ones (“I am either here or there”), which no Kalman variant can. Much more expensive.

- **Factor graph / pose graph optimisation** — store all poses and constraints and optimise them together, repeatedly relinearizing. This is what modern production SLAM systems use (GTSAM, g2o, Ceres).

17. Sensor fusion

17.1 The idea

Sensor fusion is combining data from several different sensors to get an answer better than any one of them could give alone.

The reason it works is that different sensors have *different, independent weaknesses*:

Sensor	Good at	Bad at
Lidar	Precise distance, works in darkness	Rain, fog, dust; no colour
Camera	Colour, classification, rich detail	Poor absolute distance; needs light
IMU	Very fast updates; needs nothing external	Drifts badly over seconds
Wheel odometry	Accurate over short distances	Wheel slip destroys it
GPS	No drift, absolute position	Slow, unavailable indoors, metre-level error

Look at the IMU and GPS rows: the IMU is fast but drifts, the GPS is slow but never drifts. Fuse them and you get fast *and* drift-free. Neither could do that alone. This specific pairing is what almost every aircraft and phone in the world runs.

17.2 Why the Kalman filter is the natural tool

Sensor fusion needs a principled way to decide how much to trust each sensor — and that is precisely what the Kalman gain computes, automatically, from the covariances. Each sensor's noise R says how much it deserves to be believed, and the filter does the weighting for you.

Better still, the filter handles sensors that arrive at different rates and measure different things. Each sensor gets its own H and its own R ; when a measurement arrives, you run an update with that sensor's matrices. Sensors that are silent this cycle are simply skipped. Nothing about the structure has to change.

17.3 What we fuse on the car

Our EKF combines three sources:

1. **Velocity** (V_x, V_y) from an upstream state-estimation node — drives the prediction step.
2. **Yaw** from that same node, itself derived from IMU and odometry.
3. **Cone detections** from the lidar pipeline — drive the update step.

Each contributes what it is good at: the velocity carries the estimate smoothly forward between detections, and the cones stop it drifting.

The yaw arrives from an upstream node that has already fused the IMU and the wheel odometry, so by the time it reaches this filter it is itself a fused quantity.

Part III — The implementation

18. The system

18.1 The task

Everything so far has been background. Now we use it for something real: **localizing a Formula Student race car on a track it has never seen, using only cones as landmarks.**

18.2 The tools

Tool	Role
Python	Implementation language
ROS (Robot Operating System)	Middleware — lets separate programs (nodes) exchange data over named topics
Gazebo	Physics simulator, so we could test without risking the real car
NumPy	Matrix mathematics

Tool	Role
RViz	Visualisation of the car, the cones and the estimated path

A word on **ROS**, since everything in the code is shaped by it. ROS lets you split a robot's software into independent programs called **nodes** that talk to each other by publishing and subscribing to named channels called **topics**. Our EKF is one node. It *subscribes* to topics carrying cone detections and velocities, and *publishes* topics carrying the estimated path and map. The big advantage is decoupling: the lidar processing node does not need to know anything about the filter, and either can be replaced independently.

18.3 What flows in and out

Inputs (subscribed topics):

Topic	Type	Carries
/centroid_lidar_cones_marker	MarkerArray	Detected cone positions, relative to the car
/vel_ekf	Float32MultiArray	V_x , V_y , heading, yaw
/sensor_imu_hector	Imu	Angular velocity
/robot_control/command	AckermannDriveStamped	Commanded speed and steering

Outputs (published topics):

Topic	Type	Carries
/yassers_path	Path	The estimated trajectory of the car
/global_map_yasser	MarkerArray	The estimated cone map, for RViz

18.4 Coordinate frames

Three frames matter, and mixing them up is a classic source of bugs:

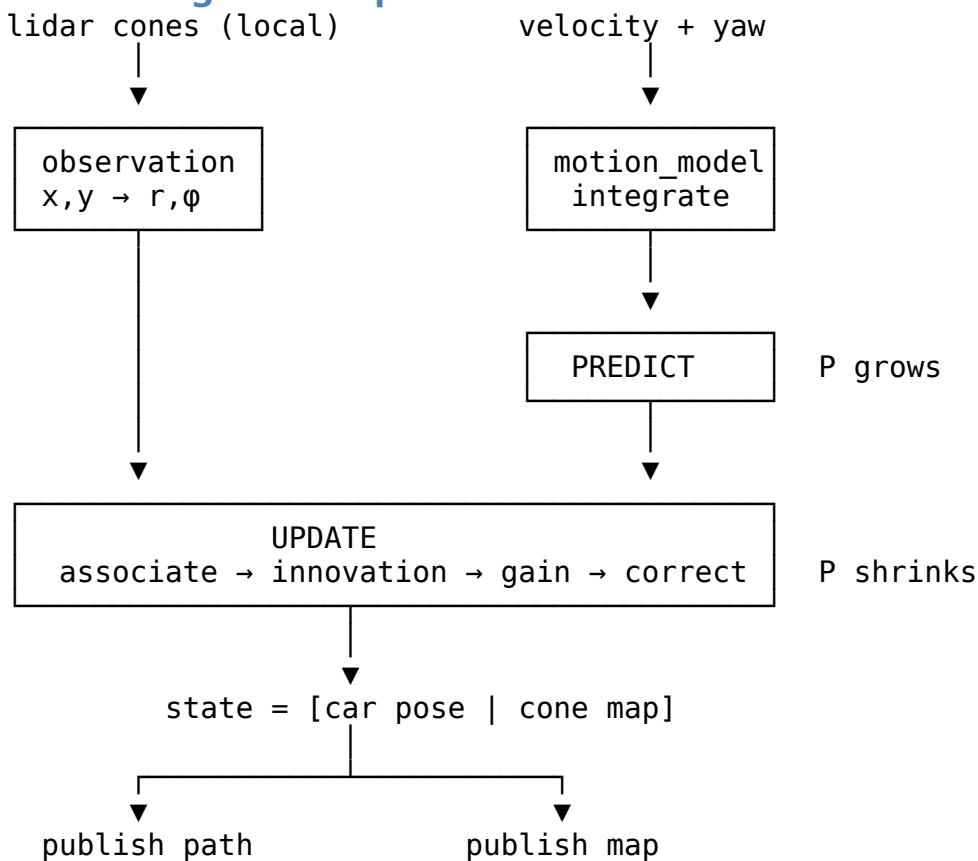
- **base_link** — attached to the car. The lidar reports cones in this frame: “4 metres ahead and 1 metre to my left.”
- **odom** — a fixed frame the car's odometry accumulates in. Smooth and continuous, but drifts.

- **map** — the global frame we want everything in. Does not drift, but can jump when the filter makes a correction.

The standard ROS convention is `map → odom → base_link`, where the SLAM system publishes the `map → odom` transform representing the accumulated drift correction. Our filter estimates the car's pose in `map` and publishes the corresponding transform.

The key conversion in the filter is from **local** measurements (relative to the car) to **global** positions (in the map). That is what `calc_LM_Pos()` does in §19.16, and getting it right requires the car's heading — which is why yaw is in the state vector.

18.5 The design in one picture



19. The code, function by function

The complete source is in `final_ekf.py` in this repository. I go through it here in the order that makes the logic easiest to follow, which is not the order it appears in the file.

19.1 Setting everything up

Execution starts at the bottom of the file:

```
642 if __name__ == '__main__':
643     rospy.init_node("final_ekf")
644
645     global oath ,dt , id ,map2
646     id =0
647     map2=MarkerArray()
648     oath = Path()
649     # EKF state covariance
650     Cx = np.diag([0.25,0.25, np.deg2rad(10.0)]) ** 2
651     R = np.diag([0.25, np.deg2rad(5.0)]) ** 2 # Measurement Noise
652     map_x2=[]
653     map_y2=[]
654
655     pub=rospy.Publisher("/yassers_path",Path,queue_size=1)
656     pub2=rospy.Publisher("/global_map_yasser",MarkerArray, queue_size=0)
657     pub3=rospy.Publisher("/global_map_centroid",MarkerArray, queue_size=0)
658
659     oath.header.frame_id="map"
660     MAX_RANGE = 6.0 # maximum observation range
661     M_DIST_TH = 1.0 # Threshold of Mahalanobis distance for data association.
662     STATE_SIZE = 3 # State size [x,y,yaw]
663     LM_SIZE = 2 # LM state size [x,y]
664     call_backs_data=callbacks_data()
665     try:
666         main(call_backs_data)
667     except rospy.ROSInterruptException:
668         pass
```

The start of execution

Line by line:

- `rospy.init_node("final_ekf")` registers this program with ROS under the name `final_ekf`. Nothing else works until this is done.
- **Cx** is the **process noise** for the pose — the *Q* of Chapter 8. As explained in §8.4, it says “each prediction may be off by about 25 cm in position and 10° in heading.” This is a primary tuning knob.
- **MAX_RANGE = 6.0** — we ignore any cone detected more than 6 m away. Beyond that our lidar’s cone detections became unreliable, and a bad detection is worse than no detection. **This is a real engineering decision with a real cost:** ignoring distant cones means fewer measurements per cycle and slightly weaker corrections, but it protects the filter from garbage input. That trade is usually worth making.
- **M_DIST_TH = 1.0** — the data-association threshold, discussed in §19.13.
- **STATE_SIZE = 3** and **LM_SIZE = 2** — the block sizes from §7.3.

19.2 Receiving data: the `callbacks_data` class

ROS delivers data asynchronously — messages arrive whenever they arrive. The standard pattern is to register **callback** functions that ROS calls on arrival, and have them store the data somewhere the main loop can read it. That container is this class:

```
28 class callbacks_data:
29     def __init__(self):
30         """This is the constructor function where every object of the class will get the same but can change values"""
31         rospy.Subscriber("/centroid_lidar_cones_marker", MarkerArray, callback=self.cones_callback) #subscribes to lidar detections
32         rospy.Subscriber("/robot_control/command", AckermannDriveStamped, self.control_cb) #subscribes to control topic
33         rospy.Subscriber("/sensor_imu_hector", Imu, self.imu_cb) #subscribes to IMU sensor
34         #rospy.Subscriber("/ground_truth/state_raw", Odometry, self.odom_cb)
35
36         rospy.Subscriber("/vel_ekf", Float32MultiArray, self.odom_callback)
37         #subscribes to Odometry topic
38
39         self.registered_map = False
40         self.control_angle = 0
41         self.control_velocity = 0
42         self.landmark_x=[]
43         self.landmark_y=[]
44         self.landmark_color=[]
45         #self.map_x = []
46         #self.map_y = []
47         #self.map_color = []
48
49         self.Velocity_x = 0
50         self.Velocity_y = 0
51         self.position_x = 0
52         self.position_y = 0
53         self.odom_orientation = 0
54         self.imu_header = 'odom'
55         self.angular_velocity = 0
56         self.Vx = 0
57         self.Vy = 0
58         self.yaw = 0
59         self.tf_listener = tf.TransformListener()
60         self.tf_broadcaster = tf.TransformBroadcaster()
61         self.tf_br = tf.TransformBroadcaster()
62         # the name of the map coordinate frame
63         self.map_frame = "map"
64         # the name of the odometry coordinate frame
65         self.odom_frame = "odom"
66         # the frame of the robot base
67         self.base_frame = "base_link"
```

The constructor of the `callbacks` class

The `__init__` method is the **constructor** — it runs exactly once, when the object is created. Two things happen in it.

First, we register all the subscribers. Second, we initialise every variable the callbacks will write to. **That second part is not optional decoration.** The main loop starts running immediately, but messages may not have arrived yet. If `self.Vx` did not exist, the first loop iteration would crash with an `AttributeError`. Initialising to zero means the filter starts from a defined state and simply predicts no motion until real data arrives.

19.3 The callbacks

```
70 def cones_callback(self, cone_detections):
71
72     self.landmark_x = []
73     self.landmark_y = []
74     self.landmark_color = []
75
76     for cone in cone_detections.markers:
77
78         self.landmark_x.append(cone.pose.position.x+1.5)
79         self.landmark_y.append(cone.pose.position.y)
80
81         r=cone.color.r
82         g=cone.color.g
83         b=cone.color.b
84         color=""
85         if r == 255 and g == 255 and b == 0 :
86             color="yellow_cone"
87         elif r == 0 and g == 0 and b == 255 :
88             color="blue_cone"
89         elif r == 255 and g == 0 and b == 0 :
90             color="orange_cone"
91         self.landmark_color.append(color)
92
93
94 def control_cb(self, control_data): #For control model
95
96     self.control_angle = control_data.drive.steering_angle
97     self.control_velocity = control_data.drive.speed
98
99
100 def imu_cb(self, imu_data):
101
102     self.angular_velocity = imu_data.angular_velocity.z
103     self.imu_header=imu_data.header.frame_id
104
105 def odom_callback(self, odom_msg):
106     self.Vx = odom_msg.data[0]
107     self.Vy = odom_msg.data[1]
108     self.heading = odom_msg.data[2]
109     self.yaw = odom_msg.data[3]
110     # print(self.Vx)
```

The callback functions

Note that each cone callback **clears the lists first**. This is deliberate and important: the lists must contain *only what the car can see right now*, not an accumulation of everything ever seen. The persistent memory of cones lives in the filter's state vector, not here.

Two things worth flagging:

- The + 1.5 added to every cone's x shifts the measurement from where the lidar is mounted to the car's reference point. Without it every cone in the map would sit 1.5 m too close to the car.

- The **cone colour** is parsed and stored. In Formula Student the track is marked with blue cones on one side and yellow on the other, so the colour carries useful information about which side of the track a cone belongs to.

19.4 The main loop

```

595 def main(call_backs_data):
596     "This is the main function where everything takes place"
597
598     global start_time
599     start_time = rospy.get_rostime().to_sec()-0.000001
600     # State Vector [x y yaw v]'
601     xEst = np.zeros((STATE_SIZE, 1))
602     PEst = np.eye(STATE_SIZE)
603     # history
604     hxEst = xEst
605     while not rospy.is_shutdown():
606         global dt
607         dt= rospy.get_rostime().to_sec() - start_time
608         start_time = rospy.get_rostime().to_sec()
609
610         #u = calc_input(call_backs_data)
611
612         z= observation(call_backs_data)
613         xEst, PEst = ekf_slam(xEst, PEst, z,call_backs_data)
614         fix_map_to_odom_transform(call_backs_data,xEst)
615         broadcast_last_transform(call_backs_data)
616         x_state = xEst[0:STATE_SIZE]
617         publish_path(xEst[:3,0])
618         #saving(xEst)
619
620
621     #publishing_cones(z)
622
623     # store data history
624     hxEst_x = []
625     hxEst_y = []
626     hxEst_w = []
627     hxEst_x.append(x_state[0])
628     hxEst_y.append(x_state[1])
629     hxEst_w.append(x_state[2])

```

The main function

Initialisation. The state starts as the zero vector:

$$\hat{x}_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} P_0 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

We *define* the car’s starting position as the origin of the map. This is legitimate — SLAM recovers the map’s shape, not its absolute position on Earth, so we may as well anchor it wherever the car happened to start. $P_0=I$ says “roughly 1 m and 1 rad of initial uncertainty”, which is somewhat arbitrary; strictly, since we *define* this pose as the origin, P_0 could be zero.

Note the state has only 3 rows at this point. It will grow as cones are discovered.

The timestep. dt is measured as the actual elapsed wall-clock time since the last iteration, rather than assumed constant. This is the honest thing to do — the loop does not run at a perfectly fixed rate, and using the true elapsed time keeps the motion model accurate. The downside is that dt becomes variable and unpredictable, and a hiccup (a long garbage-collection pause, a scheduling delay) produces a large dt , a large predicted movement, and a linearization over a large interval — which, per Chapter 14, is exactly when the EKF is least reliable.

Clamping dt to a sane maximum would be a cheap safeguard.

The cycle is then exactly Chapter 10: observe, filter, transform, publish. Forever, until ROS shuts down.

19.5 observation() — from Cartesian to polar

The lidar gives us cone positions as (x, y) offsets from the car. The filter’s measurement model works in **range and bearing**. This function converts.

```

290 def observation(Call_backs_data):
291     """
292     :param xTrue: the true pose of the system
293     :param xd: the current noisy estimate of the system
294     :param u: the current control input
295     :param RFID: the true position of the landmarks
296
297     :returns: Computes the true position, observations, dead reckoning (noisy) position,
298             and noisy control function
299     """
300     # add noise to gps x-y
301     z = np.zeros((0, 3))
302
303     for i in range(len(Call_backs_data.landmark_x)): # Test all beacons, only add the ones we can see (within MAX_RANGE)
304
305         dx = Call_backs_data.landmark_x[i]
306         dy = Call_backs_data.landmark_y[i]
307         d = math.sqrt(dx**2 + dy**2)
308         angle = pi_2_pi(math.atan2(dy, dx))
309         if d <= MAX_RANGE:
310             dn = d # add noise
311             anglen = angle # add noise
312             zi = np.array([dn, anglen, i])
313             z = np.vstack((z, zi))
314     return z

```

The observation function

The mathematics is just polar conversion:

$$d = \sqrt{\Delta x^2 + \Delta y^2} \quad \alpha = \arctan\left(\frac{\Delta y}{\Delta x}\right)$$

Why polar? Because it matches what the sensor physically does. A lidar measures a distance along a direction; range and bearing *are* its native output, and their errors are close to independent in that form. Expressing the measurement in the sensor's natural coordinates makes the noise model R much easier to specify honestly.

Note the use of `math.atan2(dy, dx)` rather than `atan(dy/dx)`. This matters: `atan2` takes both arguments separately, so it knows which quadrant the cone is in and returns an angle over the full $(-\pi, \pi]$ range. Plain `atan` would map a cone directly behind the car to the same angle as one directly ahead. **Always use `atan2` for bearings.**

Each accepted cone becomes a row [distance, angle, index], and the rows stack into the measurement matrix:

$$z = \begin{bmatrix} d_1 & \alpha_1 & 1 \\ d_2 & \alpha_2 & 2 \\ \vdots & \vdots & \vdots \end{bmatrix}$$

z is rebuilt from empty on every call, because it holds only what is visible right now.

19.6 `ekf_slam()` — the two steps

```
452 def ekf_slam(xEst, PEst, z, call_backs_data):
453     """
454     Performs an iteration of EKF SLAM from the available information.
455
456     :param xEst: the belief in last position
457     :param PEst: the uncertainty in last position
458     :param u:    the control function applied to the last position
459     :param z:    measurements at this step
460     :returns:    the next estimated position and associated covariance
461     """
462     S = STATE_SIZE
463
464     # Predict
465     xEst, PEst, G, Fx = predict(xEst, PEst, call_backs_data)
466     initP = np.eye(2)
467
468     # Update
469     xEst, PEst = update(xEst, PEst, z, initP)
470
471     return xEst, PEst
```

The `ekf_slam` function

The whole of Chapter 16 in five lines: predict, then update. `initP` is the starting covariance assigned to a newly discovered cone — the identity matrix, meaning “1 m² of variance in each axis until we learn better.”

19.7 `predict()` — moving the estimate forward

```

481 def predict(xEst, PEst, call_backs_data):
482     """
483     Performs the prediction step of EKF SLAM
484
485     :param xEst: nx1 state vector
486     :param PEst: nxn covariance matrix
487     :param u: 2x1 control vector
488     :returns: predicted state vector, predicted covariance, jacobian of control vector, transition fx
489     """
490     S = STATE_SIZE
491     G, Fx = jacob_motion(xEst, call_backs_data)
492     xEst[0:S] = motion_model(xEst[0:S], call_backs_data)
493
494     # Fx maps the pose block into the full state
495     # sigma = G*sigma*G.T + Noise
496     PEst = np.dot( G , np.dot(PEst , G.T )) + np.dot( Fx.T , np.dot(Cx , Fx) )
497
498     return xEst, PEst, G, Fx

```

The prediction function

This implements:

$$\hat{x}^- = f(\hat{x})P^- = GPG^T + F_x^T C_x F_x$$

The state goes through the exact non-linear model; the covariance goes through the Jacobian. Exactly §16.1.

The $F_x^T C_x F_x$ term deserves a note. C_x is only 3×3 — it describes noise on the car’s motion. But P may be 43×43. F_x is a 3× n matrix that is the identity in its first three columns and zero elsewhere, so sandwiching C_x between F_x^T and F_x **places the 3×3 noise block into the top-left corner of an $n \times n$ matrix of zeros**. It is a way of saying: *driving adds uncertainty to the car’s pose and to nothing else*. Which is correct — the cones do not move when the car drives.

19.8 jacob_motion() — the motion Jacobian

```
242 def jacob_motion(x, call_backs_data):
243     """
244     Calculates the jacobian of motion model.
245
246     :param x: The state, including the estimated position of the system
247     :param u: The control function
248     :returns: G: Jacobian
249               Fx: STATE_SIZE x (STATE_SIZE + 2 * num_landmarks) matrix where the left side is an identity matrix
250     """
251
252     # [eye(3) [0 x y; 0 x y; 0 x y]]
253     v=math.sqrt((call_backs_data.Vx**2)+(call_backs_data.Vy**2))
254     Fx = np.hstack((np.eye(STATE_SIZE), np.zeros(
255         (STATE_SIZE, LM_SIZE * calc_n_LM(x)))))
256     global dt
257
258     jF = np.array([[0.0, 0.0, -dt * v * math.sin(x[2, 0])],
259                   [0.0, 0.0, dt * v * math.cos(x[2, 0])],
260                   [0.0, 0.0, 0.0]])
261
262     G = np.eye(len(x)) + np.dot(np.dot(Fx.T, jF), Fx)
263     return G, Fx
```

The Jacobian of motion function

So:

$$F_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} G = \begin{bmatrix} 1 & 0 & -\Delta t v \sin \psi \\ 0 & 1 & \Delta t v \cos \psi \\ 0 & 0 & 1 \end{bmatrix}$$

Where does G come from? Differentiate the motion model with respect to each state. The model is $x_{new} = x + \Delta t \cdot v \cos \psi$, so:

- $\partial x_{new} / \partial x = 1$ — moving east by a metre moves the new position east by a metre.
- $\partial x_{new} / \partial y = 0$ — the new x does not depend on the old y.
- $\partial x_{new} / \partial \psi = -\Delta t v \sin \psi$ — **this is the interesting one**. Rotating the car changes where it ends up. This single term is the entire non-linearity of the motion model.

The bottom row is all zeros except the 1, because yaw does not depend on position.

Notice that if $v=0$ or $\Delta t=0$, then $G=I$: if the car is not moving, the mapping from old state to new state is the identity, which is exactly right.

Also note that `jacob_motion` is passed only `xEst[0:S]` — the pose, not the whole state — so `calc_n_LM()` sees a length-3 vector and returns 0. F_x is therefore 3×3 and G is 3×3 , consistent with the code updating only the 3×3 block of P .

19.9 motion_model() — the physics

```
434 def motion_model(x, call_backs_data):
435     """
436     Computes the motion model based on current state and input function.
437
438     :param x: 3x1 pose estimation
439     :param u: 2x1 control input [v; w]
440     :returns: the resulting state after the control function is applied
441     """
442     global dt
443     x[0] = x[0] + (call_backs_data.Vx)*dt # phi-math.pi/2 small track
444     x[1] = x[1] + (call_backs_data.Vy)*dt
445     #x[2] = x[2] + phi
446     x[2] = call_backs_data.yaw
447     return x
```

The motion model function

This is a **kinematic** model: it describes motion in terms of velocities and geometry only. It does not model forces, tyre grip, weight transfer or aerodynamics — those would make it a *dynamic* model. For a car at Formula Student autocross speeds over a 0.05 s timestep, kinematics is a reasonable approximation, and it has the great virtue of needing no vehicle parameters to be identified.

$$\begin{bmatrix} x \\ y \\ \psi \end{bmatrix}_{t+1} = \begin{bmatrix} x \\ y \\ \psi \end{bmatrix}_t + \begin{bmatrix} V_x \cdot \Delta t \\ V_y \cdot \Delta t \\ 0 \end{bmatrix}, \psi_{t+1} = \text{yaw from state estimation}$$

Two observations, one of which is significant.

First, V_x and V_y come from an upstream node and are already resolved into map-frame components, so the model integrates them directly rather than resolving body-frame speed through the heading itself.

Second, yaw is not integrated — it is taken directly from the upstream state estimator, which fuses the IMU and the wheel odometry and is better placed to produce it than this filter is. So the EKF integrates position forward and accepts heading as given.

19.10 update() — the correction step

This is the heart of the filter.

```

511 def update(xEst, PEst, z, initP):
512     """
513     Performs the update step of EKF SLAM
514
515     :param xEst:  nx1 the predicted pose of the system and the pose of the landmarks
516     :param PEst:  nxn the predicted covariance
517     :param u:      2x1 the control function
518     :param z:      the measurements read at new position
519     :param initP:  2x2 an identity matrix acting as the initial covariance
520     :returns:      the updated state and covariance for the system
521     """
522
523     for iz in range(len(z[:, 0])): # for each observation
524         minid = search_correspond_LM_ID(xEst, PEst, z[iz, 0:2]) # associate to a known landmark
525
526         nLM = calc_n_LM(xEst) # number of landmarks we currently know about
527
528         if minid == nLM: # Landmark is a NEW landmark
529             #print("New LM")
530             # Extend state and covariance matrix
531             xAug = np.vstack((xEst, calc_LM_Pos(xEst, z[iz, :], minid)))
532             PAug = np.vstack((np.hstack((PEst, np.zeros((len(xEst), LM_SIZE)))),
533                               np.hstack((np.zeros((LM_SIZE, len(xEst))), initP))))
534             xEst = xAug
535             PEst = PAug
536             #vlm=calc_LM_Pos(xEst, z[iz, :], minid)
537             #saving2(vlm)
538             lm = get_LM_Pos_from_state(xEst, minid)
539             #print(lm, " this is a cone location")
540             #publishing_cones(lm)
541             y, S, H = calc_innovation(lm, xEst, PEst, z[iz, 0:2], minid)
542
543             K = np.dot(np.dot(PEst, H.T), np.linalg.inv(S)) # Calculate Kalman Gain
544             xEst = xEst + np.dot(K, y)
545             PEst = np.dot((np.eye(len(xEst)) - np.dot(K, H)), PEst)
546
547
548             xEst[2] = pi_2_pi(xEst[2])
549
550             #print("size = ", calc_n_LM(xEst))
551
552             publishing_cones(xEst)
553             return xEst, PEst

```

The update function

The loop processes cones **one at a time**, running a complete correction for each. Processing measurements sequentially like this is standard and gives the same answer as one big batch update when the measurements are independent, while being far easier to implement and avoiding the inversion of a large matrix.

Growing the state for a new cone. When a cone has never been seen before, we extend both the state and the covariance:

$$x_{\text{aug}} = \begin{bmatrix} x \\ m_{\text{new},x} \\ m_{\text{new},y} \end{bmatrix} P_{\text{aug}} = \begin{bmatrix} P & 0 \\ 0 & \text{init } P \end{bmatrix}$$

The new cone's position is computed by `calc_LM_Pos()` (§19.15), and it gets a fresh covariance block of I with **zero correlation** to everything else — we have not yet learned any relationship between this cone and the rest of the map. Those

zeros will fill in on subsequent observations, and as Chapter 9 explained, that filling-in is what makes loop closure possible.

The correction itself is precisely §16.2:

$$K = P H^T S^{-1} \hat{x} = \hat{x} + K \tilde{y} \quad P = (I - K H) P$$

Note that the gain K is $n \times 2$ — it has a row for **every** state, including every cone. So a single cone observation nudges the car's pose *and every cone in the map*, each by an amount determined by how strongly the covariance says they are correlated. **One measurement improves the entire map.** That is the payoff of Chapter 9, and you can watch it happen numerically in Appendix B.

Finally `pi_2_pi` normalises the yaw, because the correction may have pushed it outside $(-\pi, \pi]$.

19.11 `calc_innovation()` — how wrong were we?

```
355 def calc_innovation(lm, xEst, PEst, z, LMid):
356     """
357     Calculates the innovation based on expected position and landmark position
358
359     :param lm: landmark position
360     :param xEst: estimated position/state
361     :param PEst: estimated covariance
362     :param z: read measurements
363     :param LMid: landmark id
364     :returns: returns the innovation y, and the jacobian H, and S, used to calculate the Kalman Gain
365     """
366     delta = lm - xEst[0:2]
367     q = np.dot(delta.T, delta)[0, 0]
368     zangle = math.atan2(delta[1, 0], delta[0, 0]) - xEst[2, 0]
369     zp = np.array([[math.sqrt(q), pi_2_pi(zangle)]])
370     # zp is the expected measurement based on xEst and the expected landmark position
371     #publishing_cones(lm, LMid)
372     y = (z - zp).T # y = innovation
373     y[1] = pi_2_pi(y[1])
374
375     H = jacobH(q, delta, xEst, LMid + 1)
376     S = np.dot(np.dot(H, PEst), H.T) + R
377
378     return y, S, H
```

The innovation function

This answers: *given where I think I am and where I think this cone is, what range and bearing should I be measuring? And how does that compare with what I actually measured?*

Step by step:

1. **delta** — the vector from the car to the cone: $\Delta = \begin{bmatrix} m_x - x \\ m_y - y \end{bmatrix}$. Both quantities come from the state, so this is entirely a prediction, not a measurement.
2. **q** — the squared distance, $\Delta_x^2 + \Delta_y^2$. Kept squared because both the range and the Jacobian are expressed more cleanly in terms of q .

3. **zp** — the **expected measurement** $h(\hat{x})$: range $\sqrt{q}$, and bearing $\arctan(\Delta_y/\Delta_x) - \psi$. Subtracting ψ converts a global direction into a bearing *relative to where the car is pointing*, which is what the lidar actually reports.
4. **y** — the innovation, $\tilde{y} = z - h(\hat{x})$. The surprise.
5. **S** — the innovation covariance, $HPH^T + R$.

Why `pi_2_pi` is applied to the angle difference. Suppose the expected bearing is 179° and the measured bearing is -179° . These are 2° apart in reality, but subtracting gives 358° . Without wrapping, the filter would see an enormous innovation and violently “correct” a state that was almost exactly right. **Angle wrapping is not a cosmetic detail — omitting it produces catastrophic, intermittent failures that are very hard to debug.** Any time you subtract two angles, wrap the result.

19.12 `jacobH()` — the measurement Jacobian

```

320 def jacobH(q, delta, x, i):
321     """
322     Calculates the jacobian of the measurement function
323
324     :param q:      the range from the system pose to the landmark
325     :param delta:  the difference between a landmark position and the estimated system position
326     :param x:      the state, including the estimated system position
327     :param i:      landmark id + 1
328     :returns:      the jacobian H
329     """
330     sq = math.sqrt(q)
331     G = np.array([[ -sq * delta[0], -sq * delta[1], 0, sq * delta[0], sq * delta[1]],
332                  [delta[1], -delta[0], -q, -delta[1], delta[0]]])
333
334     G = G / q
335     nLM = calc_n_LM(x)
336     F1 = np.hstack((np.eye(3), np.zeros((3, 2 * nLM))))
337     F2 = np.hstack((np.zeros((2, 3)), np.zeros((2, 2 * (i - 1))),
338                    np.eye(2), np.zeros((2, 2 * nLM - 2 * i))))
339
340     F = np.vstack((F1, F2))
341
342     H = np.dot(G, F)
343
344     return H

```

The Jacobian of measurement function

This is the most intricate function in the file, so let us take it in two halves.

Half one: the 2×5 core. This is the Jacobian of the measurement with respect to the five quantities that actually affect it — the car’s x , y , ψ , and the cone’s m_x , m_y :

$$G = \frac{1}{q} \begin{bmatrix} -\sqrt{q}\Delta_x & -\sqrt{q}\Delta_y & 0 & \sqrt{q}\Delta_x & \sqrt{q}\Delta_y \\ \Delta_y & -\Delta_x & -q & -\Delta_y & \Delta_x \end{bmatrix}$$

Reading it with §15.3’s rule (row = output, column = input):

- **Top row** — how **range** changes. Moving the car towards the cone decreases range; moving the cone away increases it. Hence the sign flip between the first two and last two columns. The third entry is **0**: *rotating the car does not change how far away the cone is*. That zero is a good sanity check on the whole derivation.
- **Bottom row** — how **bearing** changes. The third entry is $-q/q = -1$: rotating the car by one radian changes the relative bearing by exactly minus one radian. Also exactly right, and another useful check.

This is the same 2×2 range-bearing Jacobian we derived in §15.5, extended to account for the car's pose as well as the cone's position.

Half two: the F scattering matrix. The core is 2×5 , but H must be $2 \times n$ to multiply against an $n \times n$ covariance. F does the placement:

$$F_1 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & \cdots \\ 0 & 1 & 0 & 0 & 0 & \cdots \\ 0 & 0 & 1 & 0 & 0 & \cdots \end{bmatrix}, F_2 = \begin{bmatrix} 0 & 0 & 0 & \cdots & 1 & 0 & \cdots \\ 0 & 0 & 0 & \cdots & 0 & 1 & \cdots \end{bmatrix}$$

F_1 selects the three pose columns; F_2 has its 2×2 identity positioned at the slot belonging to **this specific cone**, with zeros everywhere else. Stacking them and multiplying gives a $2 \times n$ matrix that is zero except in the three pose columns and the two columns of the cone being observed.

And that encodes something true and important: observing cone #7 tells you directly about the car's pose and about cone #7, and about nothing else *directly*. Every other cone is affected only *indirectly*, through the correlations stored in P . The structure of H is the mathematical statement of "I can only see what I can see."

19.13 search_correspond_LM_ID() — data association

```
164 def search_correspond_LM_ID(xAug, PAug, zi):
165     """
166     Landmark association with Mahalanobis distance.
167
168     If this landmark is at least M_DIST_TH units away from all known landmarks,
169     it is a NEW landmark.
170
171     :param xAug: The estimated state
172     :param PAug: The estimated covariance
173     :param zi: the read measurements of specific landmark
174     :returns: landmark id
175     """
176
177     nLM = calc_n_LM(xAug)
178
179     mdist = []
180
181     for i in range(nLM):
182         lm = get_LM_Pos_from_state(xAug, i)
183         y, S, H = calc_innovation(lm, xAug, PAug, zi, i)
184         mdist.append(math.sqrt(np.dot(np.dot(y.T, np.linalg.inv(S)), y)[0,0]))
185
186     mdist.append(M_DIST_TH) # new landmark
187
188     minid = mdist.index(min(mdist))
189
190     return minid
```

The data association function

The problem this solves. The lidar reports “there is a cone 4 metres ahead.” It does not report *which* cone. Before we can use that measurement to correct anything, we must decide: is this a cone we already have in the map, or a new one? Getting this wrong is the single most damaging error possible in SLAM — associate a measurement with the wrong cone and you inject a large, confident, completely false correction.

The method. For every cone already in the state, compute what we *would* measure if the detection were that cone, and see how far off it is. But rather than plain distance, the code uses the **Mahalanobis distance**:

$$d_M = \sqrt{\tilde{y}^T S^{-1} \tilde{y}}$$

Why not ordinary distance? Because ordinary distance treats all disagreement equally, and it should not be. If we are very uncertain about a cone’s position, a 50 cm discrepancy is unremarkable. If we are very confident about it, the same 50 cm is damning. The Mahalanobis distance divides the discrepancy by the uncertainty (S^{-1}), producing a measure in units of *standard deviations*. It effectively asks: “**how surprising is this disagreement, given how sure I was?**”

This makes d_M **dimensionless**, which is why a single threshold can sensibly compare a discrepancy that mixes metres and radians.

The trick with the threshold. The neat part of this implementation is the line `mdist.append(M_DIST_TH)`. The threshold is appended to the list of distances as if it were one more candidate, and then we simply take the minimum. So:

- If some real cone scores below the threshold, it wins and we get its index → **existing cone**.
- If every real cone scores above the threshold, the appended threshold wins. Its position in the list is `nLM` → and `update()` reads `minid == nLM` as → **new cone**.

Both the “which cone” and the “is it new” decisions come out of one `min()`. It is a genuinely elegant piece of code, and it comes from the PythonRobotics implementation this work builds on.

Worked through with numbers. Suppose `mdist` after comparison is `[1.2, 0.5, 0.1, 0.9]` and we append the threshold to get `[1.2, 0.5, 0.1, 0.9, 1.0]`. The minimum is 0.1 at index 2, and `2 ≠ 4`, so this is **cone number 2**, already known. But if the list were `[1.2, 1.5, 1.1, 1.9, 1.0]`, the minimum is the appended 1.0 at index 4, and `4 = nLM`, so this is a **new cone** and gets added to the state.

And at the very start, when `nLM = 0`, the loop does not execute at all, `mdist` is just `[1.0]`, `minid = 0`, and `nLM = 0` — so the first cone we ever see is correctly classified as new.

On the threshold value. `M_DIST_TH = 1.0` means “if the discrepancy exceeds one standard deviation, call it a new cone.” That is deliberately tight. Statistically a correct match exceeds 1σ about 32% of the time, so this setting will occasionally create a duplicate cone rather than risk matching the wrong one — and that is the safer direction to err, because a spurious duplicate is far less damaging than a wrong association.

19.14 The small helpers

`calc_n_LM()` — how many cones are in the state:

```

273 def calc_n_LM(x):
274     """
275     Calculates the number of landmarks currently tracked in the state
276     :param x: the state
277     :returns: the number of landmarks n
278     """
279     n = int((len(x) - STATE_SIZE) / LM_SIZE)
280     return n

```

The number of landmarks function

Total length, minus the 3 rows for the car, divided by 2 rows per cone. A 43-row state means $(43 - 3)/2 = 20$ cones.

get_LM_Pos_from_state() — pull cone number *ind* out of the state:

```

196 def get_LM_Pos_from_state(x, ind):
197     """
198     Returns the position of a given landmark
199
200     :param x: The state containing all landmark positions
201     :param ind: landmark id
202     :returns: The position of the landmark
203     """
204     lm = x[STATE_SIZE + LM_SIZE * ind: STATE_SIZE + LM_SIZE * (ind + 1), :]
205
206     return lm

```

The index of a landmark in the state function

Cone 0 lives at rows 3–4, cone 1 at rows 5–6, cone 2 at rows 7–8. The formula is just that indexing arithmetic.

pi_2_pi() — wrap an angle:

```

158 def pi_2_pi(angle):
159     return (angle + math.pi) % (2 * math.pi) - math.pi

```

The pi_2_pi function

This maps **any** angle into the range $[-\pi, \pi]$. Feed it 3.5π and you get -0.5π ; feed it -7 rad and you get -0.717 rad (which is the same direction, since $-7 + 2\pi = -0.717$). The trick is the modulo: shift up by π , take the remainder against 2π (which lands in $[0, 2\pi]$), then shift back down.

calc_LM_Pos() — where is a newly seen cone, in global coordinates?

```

214 def calc_LM_Pos(x, z, id):
215     """
216     Calculates the pose in the world coordinate frame of a landmark at the given measurement.
217
218     :param x: [x; y; theta]
219     :param z: [range; bearing]
220     :returns: [x; y] for given measurement
221     """
222     zp = np.zeros((2, 1))
223
224     zp[0, 0] = x[0, 0] + z[0] * math.cos(x[2, 0] + z[1])
225     zp[1, 0] = x[1, 0] + z[0] * math.sin(x[2, 0] + z[1])
226
227     #zp[0, 0] = x[0, 0] + z[0, 0] * math.cos(x[2, 0] + z[0, 1])
228     #zp[1, 0] = x[1, 0] + z[0, 0] * math.sin(x[2, 0] + z[0, 1])
229
230     return zp

```

Calculating the pose of a landmark globally

$$z_p = \begin{bmatrix} x_{\text{car}} + d \cos(\psi + \alpha) \\ y_{\text{car}} + d \sin(\psi + \alpha) \end{bmatrix}$$

where d is the measured range and α the measured bearing. The measurement is relative to the car, but the map needs global coordinates, so we start at the car's position and step d metres in the direction $\psi + \alpha$ — the car's heading plus the cone's bearing relative to that heading. This is the inverse of the measurement model in §19.11.

Note that this is the one place where an error in the car's pose is baked directly into the map. If the pose is wrong when we first see a cone, the cone goes into the map wrong — and this is exactly the correlation that Chapter 9 said the covariance matrix must remember.

19.15 Output and housekeeping

publish_path() appends the current pose to a growing Path message and publishes it, so RViz can draw the trajectory.

publishing_cones() builds a MarkerArray of cone meshes for RViz.

saving(), **saving2()**, **savingP()** write the map and path to CSV files so they can be plotted and analysed offline. Being able to plot a run afterwards is genuinely valuable — most of the tuning insight in this project came from staring at those plots rather than at live RViz.

fix_map_to_odom_transform() and **broadcast_last_transform()** publish the ROS transform relating the map frame to the car, so that the rest of the stack can consume the filter's output through the standard tf system rather than through a custom topic.

```
631 def publish_path(data_x):  
632     poses= PoseStamped()  
633     poses.header.frame_id="map"  
634     poses.pose.position.x=data_x[0]  
635     poses.pose.position.y=data_x[1]  
636     poses.pose.orientation.w=data_x[2]  
637     global oath  
638     oath.poses.append(poses)  
639     pub.publish(oath)  
640     #print("path published")
```

The path publishing function

```

114 def publishing_cones(data):
115
116     " we will only need positions ,header ,scales , and color"
117
118
119     x=0
120     map_x=[]
121     map_y=[]
122     global map2
123
124     map2=MarkerArray()
125
126     for i in range(calc_n_LM(data)):
127
128         map_x.append(data[STATE_SIZE + i * 2,0])
129         map_y.append(data[STATE_SIZE+ i * 2 + 1,0])
130
131     #print("in map viz",len(map_x))
132     while x < len(map_x):
133
134         map_marker=Marker()
135         map_marker.header.frame_id = "map"
136         map_marker.action = Marker.ADD
137         map_marker.pose.position.x = map_x[x]
138         map_marker.pose.position.y = map_y[x]
139         map_marker.pose.position.z = 0
140         map_marker.pose.orientation.w = 1
141         map_marker.scale.x = 1
142         map_marker.scale.y = 1
143         map_marker.scale.z = 1
144         map_marker.color.a = 1
145         map_marker.id = x
146         map_marker.mesh_resource = "package://aamfsd_description/meshes/cone_blue.dae"
147         map_marker.type = Marker.MESH_RESOURCE
148         map_marker.mesh_use_embedded_materials = True
149         map2.markers.append(map_marker)
150         x+=1
151
152
153     #print("published cone ",x," number ",calc_n_LM(data))
154     pub2.publish(map2)
155     saving(map_x,map_y)

```

The cone publishing function

```

558 def saving(x,y):
559     fulldata=[[],[]]
560     for i in range(len(x)):
561         mydata=[x[i],y[i]]
562         fulldata.append(mydata)
563         i+=1
564     filename='/home/yasser/map.csv'
565     with open(filename, 'w') as csvfile:
566         csvwriter = csv.writer(csvfile)
567         csvwriter.writerows(fulldata)
568
569 def saving2(x):
570     fulldata=[[],[]]
571
572     for i in range(len(x)):
573         mydata=[x[0,0],x[1,0]]
574         fulldata.append(mydata)
575         i+=1
576     filename='/home/yasser/map_centroid.csv'
577     with open(filename, 'w') as csvfile:
578         csvwriter = csv.writer(csvfile)
579         csvwriter.writerows(fulldata)
580
581 def savingP(x):
582     fulldata=[[],[],[]]
583     #print((len(x)/2))
584     for i in range(int(len(x)/2)):
585         mydata=[x[i,0],x[i+1,0],x[i+2,0]]
586         fulldata.append(mydata)
587         i+=1
588     filename='/home/yasser/path.csv'
589     with open(filename, 'w') as csvfile:
590         csvwriter = csv.writer(csvfile)
591         csvwriter.writerows(fulldata)

```

The functions that save the map and path to CSV

```

381 def broadcast_last_transform(call_backs_data):
382
383     """ Make sure that we are always broadcasting the last map
384     to odom transformation. This is necessary so things like
385     move_base can work properly. """
386     ##### translation and rotation galeen mnen
387     call_backs_data.tf_broadcaster.sendTransform(call_backs_data.translation,
388                                                  call_backs_data.rotation,
389                                                  rospy.get_rostime(),
390                                                  call_backs_data.base_frame,
391                                                  call_backs_data.map_frame)
392
393
394 def fix_map_to_odom_transform(call_backs_data, xEst):
395     """ This method constantly updates the offset of the map and
396     odometry coordinate systems based on the latest results from
397     the localizer """
398
399     (call_backs_data.translation, call_backs_data.rotation) = convert_pose_inverse_transform(xEst[0], xEst[1], xEst[2]) #small track +math.pi/2
400
401     p = PoseStamped(pose= convert_translation_rotation_to_pose(call_backs_data.translation, call_backs_data.rotation),
402                    header=Header(frame_id=call_backs_data.base_frame))
403
404     call_backs_data.odom_to_map = call_backs_data.tf_listener.transformPose(call_backs_data.base_frame, p)
405
406
407 def convert_translation_rotation_to_pose(translation, rotation):
408     """ Convert from representation of a pose as translation and rotation (Quaternion) tuples to a geometry_msgs/Pose message """
409     return Pose(position=Point(x=translation[0], y=translation[1], z=translation[2]), orientation=Quaternion(x=rotation[0], y=rotation[1], z=rotation[2], w=rotation[3]))
410
411
412 def convert_pose_inverse_transform(poseX, poseY, theta):
413     """ Helper method to invert a transform (this is built into the tf C++ classes, but omitted from Python) """
414     translation = np.zeros((4,1))
415     translation[0] = poseX
416     translation[1] = poseY
417     translation[2] = 0.0
418     translation[3] = 1.0
419
420
421
422     rotation = np.transpose(rotation_matrix(0, [0,0,1]))
423     transformed_translation = rotation.dot(translation)
424
425     translation = (transformed_translation[0], transformed_translation[1], transformed_translation[2])
426     rotation = quaternion_from_matrix(rotation)
427     rotation = tf.transformations.quaternion_from_euler(0.0, 0.0, theta)
428
429
430     return (translation, rotation)

```

The frame transform helper functions

20. Running it

20.1 Requirements

- Ubuntu with ROS 1 (Melodic or Noetic)
- Python 2.7 / 3.x matching your ROS distribution
- numpy, rospy, tf, ros_numpy
- The team's custom messages (aam_common_msgs)
- Gazebo, for simulation

20.2 Launching

1. Start the simulator and the rest of the pipeline

```
roslaunch aamfsd_gazebo <your_track>.launch
```

2. Run the filter

```
roslaunch <your_package> final_ekf.py
```

3. Visualise

```
rviz
```

In RViz, add displays for /yassers_path (the estimated trajectory) and /global_map_yasser (the estimated cone map), with the fixed frame set to map.

20.3 What you should see

The car drives, and a trail of estimated poses grows behind it while cones pop into the map as they come within 6 m. The things worth watching:

- **Does the map stay stable?** Cones should stay put once placed. If they jitter or drift, data association is struggling or the noise parameters need work.
- **Are there duplicates?** Two cones appearing where there is physically one means association is failing — usually `M_DIST_TH` is too tight or the pose estimate is drifting between sightings.
- **Does the path look like the track?** The best single check. Track shape is very recognisable, and errors show up immediately to the eye.

21. Tuning

The equations are fixed; the parameters are not, and the parameters are what decide whether the filter works. This chapter is about the ones that mattered.

21.1 The fundamental trade-off

Everything comes back to one balance:

Q (process noise) says how much you distrust your model. R (measurement noise) says how much you distrust your sensors. Only their ratio matters.

That last point is worth dwelling on. Scaling both Q and R by 10 leaves the Kalman gain unchanged, and therefore leaves the state estimate unchanged — the covariance values change, but the trajectory does not. So there is no point tuning them independently as if they were separate knobs. **Fix one from physical measurement and tune the other against it.**

The preferred order: measure R honestly (park the car, stare at a stationary cone, record several hundred detections, compute the variance — exactly the Chapter 3 procedure), then tune Q until the filter behaves.

21.2 Symptoms and what they mean

Symptom	Likely cause	Fix
Estimate lags reality, slow to react	Q too small — over-trusting the model	Increase Q
Estimate jumps around,	Q too large — chasing	Decrease Q

Symptom	Likely cause	Fix
noisy	every measurement	
Ignores obviously good measurements	R too large	Decrease R
Duplicate cones in the map	M_DIST_TH too tight, or pose drifting	Increase threshold; check yaw
Cones merging into one	M_DIST_TH too loose	Decrease threshold
Sudden divergence into nonsense	Bad association, or linearization over too large a step	Check dt ; check angle wrapping
Slows down over time	State grew too large (§21.4)	Cap landmarks; prune

21.3 A diagnostic worth knowing

There is a principled way to check whether Q and R are honest, rather than tuning by eye. If the filter's noise model is correct, then the **normalised innovation squared**:

$$NIS = \tilde{y}^T S^{-1} \tilde{y}$$

should average out to the dimension of the measurement — 2 in our case, since we measure range and bearing.

- **NIS consistently above 2** → innovations are bigger than the filter expects → the filter is **over-confident**, and Q or R is too small.
- **NIS consistently below 2** → the filter is **under-confident**, and the noise values are too large.

This is the same quantity as the Mahalanobis distance from §19.13, squared, and logging its running average is a cheap and very effective health check — it turns tuning from guesswork into measurement.

21.4 Computational cost

EKF-SLAM's weakness is scaling. The covariance is $n \times n$ where $n = 3 + 2M$ for M cones, and the update touches all of it, so cost grows as $O(M^2)$.

With 30 cones ($n=63$) this is trivial. With 300 cones ($n=603$) the matrices have over 360,000 entries and every update touches all of them. Since a full Formula Student track can carry a few hundred cones, this is not a theoretical concern —

it is the practical ceiling on the approach, and it is the main reason larger systems move to graph-based SLAM.

Le Large’s thesis quantifies exactly this trade-off on real Formula Student hardware: GraphSLAM produced more accurate maps, but consumed computational resources much faster than EKF-SLAM, and he concludes that where compute is constrained, EKF-SLAM remains “a valid option” whose lower accuracy may be offset by lower CPU usage. For our compute budget, that was the right trade.

Appendix A — Notation

Symbol	Name	Meaning
μ	mean	The average; the centre of a Gaussian; our best single guess
σ^2	variance	How spread out the data is
σ	standard deviation	Square root of variance; same units as the data
$E[x]$	expected value	The long-run average of a random process
x	state vector	Everything we are estimating
$\hat{x}$	state estimate	Our current best guess (the hat means “estimated”)
$\hat{x}^-$	prior estimate	After prediction, before update
P	covariance matrix	How wrong we might be, in every direction
Q, C_x	process noise	How much we distrust the motion model
R	measurement noise	How much we distrust the sensors
F	state transition	Linear model of how the state evolves

Symbol	Name	Meaning
G	motion Jacobian	Linearized motion model (EKF)
H	measurement Jacobian	Linearized measurement model
K	Kalman gain	How much to trust the measurement vs the prediction
$\tilde{y}$	innovation	Measured minus expected — the “surprise”
S	innovation covariance	How surprised we expected to be
z	measurement	What the sensor reported
ψ	yaw	Heading angle of the car
Δt	timestep	Time between filter cycles
d_M	Mahalanobis distance	Discrepancy measured in standard deviations
q	—	Squared distance from car to landmark
M	—	Number of landmarks in the map

Appendix B — One complete cycle, by hand

Here is a full EKF-SLAM cycle with real numbers, using the textbook equations from §16.2. If you follow this all the way through, you understand the filter.

Setup

The car is at $(1.0, 0.5)$ metres with heading 0.20 rad, and it already knows about one cone, which it believes is at $(5.0, 1.0)$.

$$\hat{x} = \begin{bmatrix} 1.0 \\ 0.5 \\ 0.20 \\ 5.0 \\ 1.0 \end{bmatrix} P = \begin{bmatrix} 0.10 & 0 & 0 & 0 & 0 \\ 0 & 0.10 & 0 & 0 & 0 \\ 0 & 0 & 0.05 & 0 & 0 \\ 0 & 0 & 0 & 0.30 & 0 \\ 0 & 0 & 0 & 0 & 0.30 \end{bmatrix}$$

The car is travelling at $v=2$ m/s, the timestep is $\Delta t=0.1$ s, and the noise values are:

$$C_x = \text{diag}(0.0625, 0.0625, 0.03046) \quad R = \text{diag}(0.10, 0.02)$$

Step 1 — Predict the state

$$\Delta x = \Delta t \cdot v \cos \psi = 0.1 \times 2 \times \cos(0.20) = 0.19601$$

$$\Delta y = \Delta t \cdot v \sin \psi = 0.1 \times 2 \times \sin(0.20) = 0.03973$$

$$\hat{x}^- = \begin{bmatrix} 1.19601 \\ 0.53973 \\ 0.20 \\ 5.0 \\ 1.0 \end{bmatrix}$$

The car moved about 20 cm. The cone did not move — correct, cones do not move.

Step 2 — Predict the covariance

$$G = \begin{bmatrix} 1 & 0 & -0.0397 & 0 & 0 \\ 0 & 1 & 0.1960 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$P^- = G P G^T + F_x^T C_x F_x = \begin{bmatrix} 0.1626 & -0.0004 & -0.0020 & 0 & 0 \\ -0.0004 & 0.1644 & 0.0098 & 0 & 0 \\ -0.0020 & 0.0098 & 0.0805 & 0 & 0 \\ 0 & 0 & 0 & 0.30 & 0 \\ 0 & 0 & 0 & 0 & 0.30 \end{bmatrix}$$

Uncertainty grew, as it must during prediction: the x variance went from 0.10 to 0.1626. Note also that small off-diagonal terms have appeared between position and yaw — the Jacobian has coupled them, because moving while uncertain about heading makes the position error depend on the heading error.

Step 3 — Take a measurement

The lidar reports the cone at range 4.10 m, bearing 0.05 rad.

$$z = \begin{pmatrix} 4.10 \\ 0.05 \end{pmatrix}$$

Step 4 — What did we expect to measure?

$$\Delta = \begin{pmatrix} 5.0 - 1.19601 \\ 1.0 - 0.53973 \end{pmatrix} = \begin{pmatrix} 3.80399 \\ 0.46027 \end{pmatrix}$$

$$q = 3.80399^2 + 0.46027^2 = 14.6822, \sqrt{q} = 3.83173$$

$$\hat{z} = \begin{pmatrix} 3.83173 \\ \arctan(0.46027/3.80399) - 0.20 \end{pmatrix} = \begin{pmatrix} 3.83173 \\ -0.07959 \end{pmatrix}$$

Step 5 — The innovation

$$\tilde{y} = z - \hat{z} = \begin{pmatrix} 4.10 - 3.83173 \\ 0.05 - (-0.07959) \end{pmatrix} = \begin{pmatrix} 0.26827 \\ 0.12959 \end{pmatrix}$$

The cone is 27 cm further away and 0.13 rad further round than we expected. Something is slightly off — either our position, or the cone's recorded position, or sensor noise. The filter does not need to know which; it will distribute the correction according to the covariances.

Step 6 — The measurement Jacobian

$$H = \begin{pmatrix} -0.9928 & -0.1201 & 0 & 0.9928 & 0.1201 \\ 0.0313 & -0.2591 & -1 & -0.0313 & 0.2591 \end{pmatrix}$$

Check the two entries §19.12 predicted: the top-row yaw entry is exactly **0** (rotating does not change range) and the bottom-row yaw entry is exactly **-1** (rotating by one radian changes bearing by minus one radian). The Jacobian is correct.

Step 7 — Innovation covariance and gain

$$S = H P^- H^T + R = \begin{pmatrix} 0.5625 & -0.0008 \\ -0.0008 & 0.1373 \end{pmatrix}$$

$$K = P^- H^T S^{-1} = \begin{bmatrix} -0.2868 & 0.0506 \\ -0.0350 & -0.3819 \\ 0.0005 & -0.6050 \\ 0.5294 & -0.0653 \\ 0.0649 & 0.5665 \end{bmatrix}$$

K is 5×2 — a row for every state, including both cone coordinates. **This is how one observation corrects the whole state.**

(Incidentally, the Mahalanobis distance for this match is $\sqrt{\tilde{y}^T S^{-1} \tilde{y}} = 0.50$, comfortably below $M_DIST_TH = 1.0$, so \$19.13 would correctly accept this as the known cone.)

Step 8 — Correct

$$K \tilde{y} = \begin{bmatrix} -0.0704 \\ -0.0589 \\ -0.0783 \\ 0.1336 \\ 0.0908 \end{bmatrix} \Rightarrow \hat{x} = \begin{bmatrix} 1.1256 \\ 0.4809 \\ 0.1217 \\ 5.1336 \\ 1.0908 \end{bmatrix}$$

The filter moved the car back slightly, adjusted its heading, **and pushed the cone outwards** — it split the discrepancy between “I am not quite where I thought” and “the cone is not quite where I thought”, in proportion to how confident it was about each.

Step 9 — Correct the covariance

$$P = (I - K H) P^- = \begin{bmatrix} 0.1159 & -0.0033 & 0.0024 & 0.0859 & 0.0064 \\ -0.0033 & 0.1437 & -0.0219 & 0.0068 & 0.0309 \\ 0.0024 & -0.0219 & 0.0302 & -0.0058 & 0.0470 \\ 0.0859 & 0.0068 & -0.0058 & 0.1417 & -0.0140 \\ 0.0064 & 0.0309 & 0.0470 & -0.0140 & 0.2536 \end{bmatrix}$$

Two things to notice, and the second is the important one.

Uncertainty shrank everywhere. In standard deviations:

	before	after
car x	0.403 m	0.341 m
car y	0.406 m	0.379 m
car ψ	0.284 rad	0.174 rad
cone x	0.548 m	0.377 m

	before	after
cone y	0.548 m	0.504 m

The total uncertainty (trace of P) fell from 1.008 to 0.685 — a 32% reduction from a single cone sighting. Note the heading improved most in relative terms, which makes sense: a bearing measurement is very informative about heading.

Correlations appeared. The highlighted **0.0859** entries are the covariance between the car's x and the cone's x . They were zero before this update; now they are not. **The filter has learned that its belief about the car and its belief about that cone are linked.**

That single number is the whole point of Chapter 9. It means that the next time this cone is seen, the correction will flow back into the car's pose — and that is loop closure, appearing in a real matrix, from one measurement.

Appendix C — Glossary

Bayes filter — The general recursive framework for updating a belief with new evidence. The Kalman filter is the special case where everything is Gaussian and linear.

Bias — A systematic, non-random error. A sensor that always reads 5 units high has a bias. Filtering cannot remove it (§6.3).

Covariance — A measure of how two variables' errors relate. Also used loosely for the whole uncertainty matrix P .

Data association — Deciding which known landmark a new detection corresponds to, or whether it is new. §19.13.

Dead reckoning — Estimating position purely by integrating motion, with no external reference. Simple, and drifts without bound.

Divergence — When a filter's estimate runs away from reality while its covariance claims confidence. The characteristic EKF failure mode.

Drift — Slow accumulation of error over time, characteristic of integrating noisy measurements.

EKF — Extended Kalman Filter. A Kalman filter applied to non-linear systems by linearizing them with Jacobians at each step.

Gaussian / normal distribution — The bell curve. Fully described by a mean and a variance.

Innovation — Measurement minus expected measurement. The “surprise”. Everything the filter learns, it learns from this.

Jacobian — The matrix of first-order partial derivatives; the best linear approximation of a non-linear function at a point. §15.

Kalman gain — The weighting between prediction and measurement, computed automatically from the covariances. §11.3.

Landmark — A recognisable fixed feature used as a reference. Here, a traffic cone.

Linearization — Replacing a non-linear function with its tangent line near a point. §13.4.

Loop closure — Recognising a previously visited place and using it to correct accumulated drift across the whole map.

Mahalanobis distance — Distance measured in units of standard deviations, accounting for uncertainty. §19.13.

NIS — Normalised Innovation Squared. A statistic for checking whether a filter’s noise model is honest. §21.3.

Odometry — Position estimated from wheel rotation. Accurate short-term, drifts long-term.

Pose — Position and orientation together. Here, (x, y, ψ) .

Precision (statistics) — The reciprocal of variance, $1/\sigma^2$. Also called information. Precisions add when fusing. §12.3.

Process noise — Error arising because the world does not behave exactly as the model says. §8.2.

ROS — Robot Operating System. Middleware for connecting robot software components.

Sensor / measurement noise — Error in the act of measuring. §8.1.

SLAM — Simultaneous Localization And Mapping.

State vector — The list of quantities being estimated. §7.2.

Taylor series — An expansion of a function into terms built from its derivatives at a point. §14.

UKF — Unscented Kalman Filter. Handles non-linearity with sample points instead of Jacobians. §16.5.

Yaw — Rotation about the vertical axis; which way the car points.

References

Primary references for this work

1. **Le Large, N.** (2020). *A comparison of different approaches to solve the SLAM problem on a Formula Student Driverless race car*. Master's Thesis, Institute of Measurement and Control Systems, Karlsruhe Institute of Technology. Reviewer: Prof. Dr.-Ing. Christoph Stiller; Advisor: Frank Bieder, M.Sc. — *The most directly relevant reference for this project: a rigorous comparison of EKF-SLAM and GraphSLAM on real Formula Student data, evaluated against DGPS ground truth. Its conclusion — that GraphSLAM is more accurate but consumes computational resources much faster, leaving EKF-SLAM “a valid option” where compute is limited — is precisely the trade-off discussed in §21.4.*
2. **Sakai, A. et al.** *PythonRobotics: a Python code collection of robotics algorithms*. <https://atsushisakai.github.io/PythonRobotics/> — *The EKF-SLAM implementation in this project is derived from the PythonRobotics `ekf_slam.py` example, and retains its structure and function naming (`calc_n_LM`, `search_correspond_LM_ID`, `jacob_h`, the `M_DIST_TH` association trick). An outstanding resource for learning robotics algorithms from readable, runnable code. See also: Sakai, A., Ingram, D., Dinius, J., Chawla, K., Raffin, A., Paques, A. (2018). *PythonRobotics: a Python code collection of robotics algorithms*. arXiv:1808.10703.*
3. **Becker, A.** *Kalman Filter Tutorial*. <https://www.kalmanfilter.net/> — *The clearest introduction to the Kalman filter available online, building from one-dimensional examples with full numerical tables. Recommended as the next thing to read after this document.*

Books

4. **Thrun, S., Burgard, W., & Fox, D.** (2005). *Probabilistic Robotics*. MIT Press. — *The standard graduate text. Chapter 10 covers EKF-SLAM in full, including the propagation of the landmark cross-covariances.*
5. **Labbe, R.** *Kalman and Bayesian Filters in Python*.
<https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python> — *A free, interactive book of Jupyter notebooks. Exceptionally good at building intuition, with every figure generated by code you can modify.*
6. **Barfoot, T. D.** (2017). *State Estimation for Robotics*. Cambridge University Press. — *More mathematically rigorous; excellent on the geometry of rotations.*
7. **Simon, D.** (2006). *Optimal State Estimation: Kalman, H^∞ , and Nonlinear Approaches*. Wiley. — *Thorough treatment of filter variants and their failure modes.*

Foundational papers

8. **Kálmán, R. E.** (1960). “A New Approach to Linear Filtering and Prediction Problems.” *Journal of Basic Engineering*, 82(1), 35–45. — *The original paper.*
9. **Smith, R., Self, M., & Cheeseman, P.** (1990). “Estimating Uncertain Spatial Relationships in Robotics.” *Autonomous Robot Vehicles*, Springer, 167–193. — *The paper that established the EKF-SLAM formulation, including why landmark correlations must be maintained.*
10. **Julier, S. J., & Uhlmann, J. K.** (1997). “A New Extension of the Kalman Filter to Nonlinear Systems.” *Proc. AeroSense: 11th Int. Symp. Aerospace/Defence Sensing, Simulation and Controls*. — *Introduces the Unscented Kalman Filter (§16.5).*
11. **Welch, G., & Bishop, G.** (2006). *An Introduction to the Kalman Filter*. Technical Report TR 95-041, University of North Carolina at Chapel Hill. — *A concise classic; the source of the predict–update diagram style used in §16.2.*
12. **Bailey, T., & Durrant-Whyte, H.** (2006). “Simultaneous Localization and Mapping (SLAM): Part I & Part II.” *IEEE Robotics & Automation Magazine*, 13(2–3). — *The standard survey of the field.*

Formula Student Driverless

13. **Kabzan, J. et al.** (2020). “AMZ Driverless: The Full Autonomous Racing System.” *Journal of Field Robotics*, 37(7), 1267–1294. — *A complete description of a winning Formula Student driverless stack, from perception through to control.*
14. **Andresen, L. et al.** (2020). “Accurate Mapping and Planning for Autonomous Racing.” *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

Software

15. **ROS (Robot Operating System)** — <https://www.ros.org/>
16. **NumPy** — Harris, C. R. et al. (2020). “Array programming with NumPy.” *Nature*, 585, 357–362.
17. **Gazebo simulator** — <https://gazebo.org/>